Scrap

Senior Design (423-IDE401 / 424-IDE402) Core Report

Version 1.31

Milestone 1 Draft

by

Salome Orozco, Samah Keshiro, Deanna Gaber

Stevens Institute of Technology

September 24, 2026

© Stevens Institute of Technology 2026
ALL RIGHTS RESERVED

Scrap

Senior Design (423-IDE401 / 424-IDE402) Core Report

Version 1.31
Milestone 1 Draft

Salome Orozco, Samah Keshiro, Deanna Gaber
Stevens Institute of Technology
September 24, 2026

This document specifies the stakeholder needs, requirements, architecture, implementation constraints, tests, verification evidence, validation evidence, and release criteria for the Scrap. It is both a printable engineering record and the authoritative input for AI-assisted implementation. A conforming implementation may be regenerated from the complete specification, but no single chapter should be treated as sufficient on its own.

Document History

Add the newest approved change at the top of the table. Retain every approved entry so that the report's evolution and review history remain auditable.

Table 1: Document Update History

Date / Version	Author	Change	Location
October 28, 2026 / 1.31	DMG, SAK, SAO	Added the Week 9 report, covering finishing and practicing our Milestone 1 design review slide deck and presenting it, with next week's goal to finally begin tangible development on the app itself.	Appendix A: Weekly Reports
October 27, 2026 / 1.30	DMG, SAK, SAO	Closed gaps identified against the Milestone 2 team-report requirements: added a new Codes and Standards chapter consolidating every process and product-facing standard the team cites (including two new product-facing standards, WCAG 2.1 and Apple's Human Interface and App Store Review Guidelines, with their own verification rows VER-014 and VER-015); added two activity diagrams (ingredient-match search, photo-based pantry entry) to Use Cases; added four missing validation scenarios (VAL-007 through VAL-010) covering end-user stories US-03, US-04, US-06, and US-07; added a verification row for mobile responsiveness (VER-013); and added an explicit Tradeoffs and a Documented Design Iteration section to Key Constraints narrating the Milestone 1 feedback that produced REQ-F-19 and REQ-F-20 as a complete feedback-to-recheck loop.	Codes and Standards; Use Cases; Verification, Validation, and Testing; Key Constraints

Continued on next page

Date / Version	Author	Change	Location
October 24, 2026 / 1.29	DMG, SAK, SAO	Added the Week 8 report, covering our discovery that the Xcode version we downloaded (13.4.1) cannot be used for this project, and our team's ongoing decision between cloud-hosted Mac access and buying a personal Mac.	Appendix A: Weekly Reports
October 16, 2026 / 1.28	DMG, SAK, SAO	Added a Developer Requirements and Coding Standards chapter (REQ-D-01 through REQ-D-26) fixing our MVVM architecture split, namespace/folder structure, naming conventions, and UML diagrams, and added the Week 7 report covering that work and getting the department Mac working.	Developer Requirements and Coding Standards; Appendix A: Weekly Reports
October 8, 2026 / 1.27	DMG, SAK, SAO	Updated the Week 6 report: added planning our MVVM architecture, naming conventions, and UML diagrams to next week's plan and action items, and noted in Issues and Risks that our professor is reaching out to see if the program has a Mac we could borrow.	Appendix A: Weekly Reports
October 7, 2026 / 1.26	DMG, SAK, SAO	Added the Week 6 report to Appendix A, covering this week's finding that there is no solid way to use Xcode without an actual Mac, the decision to split the codebase into ScrapCore and ScrapApp to minimize reliance on shared library computers, and the resulting code scaffold.	Appendix A: Weekly Reports
October 7, 2026 / 1.25	DMG, SAK, SAO	Added a fourth risk to Risks and Assumptions covering the team's lack of personal Mac access and reliance on shared library computers, and its mitigation: splitting the codebase into a platform-independent ScrapCore package and a Mac-only SwiftUI shell (ScrapApp).	Development Plan

Continued on next page

Date / Version	Author	Change	Location
October 1, 2026 / 1.24	DMG, SAK, SAO	Added the Week 5 report to Appendix A, covering this week's hand-drawn initial screens, AI-generated sample screens and design comments, and next week's plan to look into operations-stakeholder requirements, deployment approach, and Swift-specific edge cases and constraints.	Appendix A: Weekly Reports
September 29, 2026 / 1.23	DMG, SAK, SAO	Widened the Source column of the User Stories to Requirements table so every entry ("Interviewee N," "Derived (developer/business/community)") fits on one line instead of wrapping.	Traceability Matrix
September 29, 2026 / 1.22	DMG, SAK, SAO	Added a Verification, Validation, and Testing chapter with a Verification Register, a Validation Register, and example test cases mapped to Scrap's requirements, use cases, and user stories. Added two bonus functional requirements from usability feedback gathered after the Milestone 1 class presentation — REQ-F-19 (photo-based pantry entry) and REQ-F-20 (automatic shelf-life estimation and expiration reminders) — and two corresponding use cases (UC-11, UC-12), and updated the Requirements, Use Cases, and Traceability Matrix chapters accordingly.	Verification, Validation, and Testing; Requirements; Use Cases; Traceability Matrix
September 24, 2026 / 1.21	DMG, SAK, SAO	Added citations for Apple's SwiftUI documentation, the Spoonacular API, and Supabase's iOS/SwiftUI quickstart guide to the Mobile Application, Backend, and Recipe and Ingredient Data sections of the Development Plan chapter and the bibliography.	Development Plan

Continued on next page

Date / Version	Author	Change	Location
September 24, 2026 / 1.20	DMG, SAK, SAO	Rebuilt the Timeline and Milestones section as two dated tables: biweekly fall checkpoints from September 24 through a working prototype on December 3, and monthly spring goals from January through the final report in May, with the official course milestones labeled where they fall.	Development Plan
September 24, 2026 / 1.19	DMG, SAK, SAO	Revised the Timeline and Milestones section with the team's own targets: a working prototype by early December, and a spring semester focused on beta user testing, iteration, and growing an initial user base.	Development Plan
September 24, 2026 / 1.18	DMG, SAK, SAO	Added three cited sources on food waste to the Key Drivers chapter and the bibliography; added Risks and Assumptions and Timeline and Milestones sections to the Development Plan chapter, the latter following the course's official milestone schedule from the full report template. Updated the Week 4 report to reflect this work.	Key Drivers; Development Plan; Appendix A: Weekly Reports
September 24, 2026 / 1.17	DMG, SAK, SAO	Replaced the Development Method section of the Development Plan chapter with Workspace and Communication Plan sections, detailing the team's GitHub branching workflow, shared documents, tooling, and check-in practices.	Development Plan
September 24, 2026 / 1.16	DMG, SAK, SAO	Added a Roles and Responsibilities section to Team Declaration, assigning each team member a lead role: Product & AI Integration, iOS App & Backend, and Business Strategy & Marketing.	Team Declaration
September 23, 2026 / 1.15	DMG, SAK, SAO	Added a Development Plan chapter after Key Constraints, summarizing the mobile application, backend, recipe data source, companion website, development method, and testing plan.	Development Plan

Continued on next page

Date / Version	Author	Change	Location
September 23, 2026 / 1.14	DMG, SAK, SAO	Added the Week 4 report to Appendix A, Weekly Reports. Condensed every AI Prompt Log entry to two sentences or fewer.	Appendix A: Weekly Reports; Appendix B: AI Prompt Log
September 23, 2026 / 1.13	DMG, SAK, SAO	Aligned the report with the team's technical development plan: rewrote the Requirements chapter's Budget and Data Sources constraints and the Key Constraints data-management paragraph to name the actual chosen stack (Swift/SwiftUI, Supabase, Spoonacular, Open Food Facts, Vercel/Netlify); added a bonus barcode-entry functional requirement (REQ-F-18) and a bug-tracking constraint (REQ-C-06); revised REQ-T-01, REQ-T-03, REQ-T-04, and REQ-T-06 to name the specific tools chosen, and added REQ-T-08 through REQ-T-13 (backend platform, database schema, recipe data source, third-party API resilience, beta distribution, and the companion website). Updated Use Cases and the Traceability Matrix to match.	Key Constraints; Requirements; Use Cases; Traceability Matrix
September 22, 2026 / 1.12	DMG, SAK, SAO	Trimmed all fourteen user stories to a single sentence each, removing the extra clauses added in v1.11.	User Stories
September 22, 2026 / 1.11	DMG, SAK, SAO	Rewrote the end-user stories in "As a / I want / so that" form and removed interviewee names, keeping only anonymized interview references. Added six stakeholder user stories for developers, business/product, and community moderation. Added a Technical Requirements (iOS Platform) section (REQ-T-01 through REQ-T-07) and a Content Moderation functional requirement (REQ-F-17), both traced from the new stakeholder stories. Updated the Traceability Matrix and Use Cases with the new stories and requirements.	User Stories; Requirements; Use Cases; Traceability Matrix

Continued on next page

Date / Version	Author	Change	Location
September 22, 2026 / 1.10	DMG, SAK, SAO	Corrected the document status from Milestone 0 back to Milestone 1 throughout. Changed REQ-F-07, REQ-F-13, and REQ-F-14 from "shall" to "could," marking recipe video, leftover tracking, and expiration alerts as bonus features rather than required for the minimum viable product.	Document History; Requirements
September 22, 2026 / 1.9	DMG, SAK, SAO	Added User Stories, Requirements, Use Cases, and Traceability Matrix chapters, built from the team's Week 3 stakeholder interviews and AI-assisted starting list of use cases.	User Stories; Requirements; Use Cases; Traceability Matrix
September 17, 2026 / 1.8	DMG, SAK, SAO	Corrected the document status from Milestone 1 to Milestone 0 throughout. Revised the Mission Statement and Key Drivers to state the team's four core values (easy, affordable, fun, and less wasteful) and to more strongly highlight the social aspect of Scrap.	Mission Statement; Key Drivers
September 17, 2026 / 1.7	DMG, SAK, SAO	Added the Week 3 report to Appendix A, Weekly Reports.	Appendix A: Weekly Reports
September 15, 2026 / 1.6	DMG, SAK, SAO	Removed the Template Attribution chapter from the front matter. Listed the weekly reports in Appendix A in reverse chronological order (most recent week first).	Document History; Appendix A: Weekly Reports
September 10, 2026 / 1.5	DMG, SAK, SAO	Made the Location column in this table clickable, linking each entry to its chapter or appendix. Added the GitHub repository setup to the Week 2 report.	Document History; Appendix A: Weekly Reports
September 10, 2026 / 1.4	DMG, SAK, SAO	Updated author initials throughout this table to reflect the full team. Clarified in the Week 2 report that the Key Drivers and Key Constraints chapters were started but not yet finished as of that week.	Document History; Appendix A: Weekly Reports

Continued on next page

Date / Version	Author	Change	Location
September 10, 2026 / 1.3	DMG, SAK, SAO	Added the Week 2 report to Appendix A, Weekly Reports. Removed the placeholder example rows from this table.	Appendix A: Weekly Reports; Document History
September 10, 2026 / 1.2	DMG, SAK, SAO	Added Appendix A, Weekly Reports (Week 1 report), and Appendix B, AI Prompt Log, to this Milestone 1 submission copy.	Appendix A: Weekly Reports; Appendix B: AI Prompt Log
September 10, 2026 / 1.1	DMG, SAK, SAO	Created this trimmed Milestone 1 submission copy from the full working template, containing only the front matter and the chapters completed to date.	Entire document
September 8, 2026 / 1.1	DMG, SAK, SAO	Customized the template for Scrap. Added the Team Declaration chapter (team name, members, project proposal) and the Milestone 1 content: Mission Statement, Key Drivers, and Key Constraints. Updated project metadata.	Team Declaration; Mission Statement; Key Drivers; Key Constraints
September 1, 2026 / 1.0	DDM, CDX	Adapted the system and software architecture design template as the Senior Design Core Report template. Added the complete 423-IDE401 / 424-IDE402 report structure, milestones, and appendices.	Entire report

Table of Contents

Document History iv

I Project Definition and Plan 1

1 Team Declaration 2
- 1.1 Team Name . . . 2
- 1.2 Team Members . . . 2
- 1.3 Roles and Responsibilities . . . 2
- 1.4 Project Proposal . . . 3

2 Mission Statement 4

3 Key Drivers 5

4 Key Constraints 7
- 4.1 Tradeoffs and a Documented Design Iteration . . . 8

5 Development Plan 9
- 5.1 Mobile Application . . . 9
- 5.2 Backend . . . 9
- 5.3 Recipe and Ingredient Data . . . 10
- 5.4 Companion Website . . . 10
- 5.5 Workspace . . . 10
- 5.6 Communication Plan . . . 10
- 5.7 Risks and Assumptions . . . 11
- 5.8 Timeline and Milestones . . . 11
- 5.9 Testing Plan . . . 13

6 User Stories 14
- 6.1 Interview Summary . . . 14
- 6.2 End-User Stories . . . 15
- 6.3 Other Stakeholder Stories . . . 16

7 Requirements **17**
7.1 Functional Requirements . . . 17
7.2 Non-Functional Requirements . . . 19
7.3 Constraints . . . 20
7.4 Technical Requirements . . . 21

8 Use Cases **23**
8.1 Activity Diagrams for Key Workflows . . . 28

9 Traceability Matrix **31**
9.1 User Stories to Requirements . . . 31
9.2 Functional Requirements to Stories and Use Cases . . . 32
9.3 Non-Functional Requirements to Stories and Use Cases . . . 33
9.4 Constraints to Stories and Use Cases . . . 33
9.5 Technical Requirements to Stories and Use Cases . . . 34
9.6 Use Cases to Requirements . . . 35

10 Verification, Validation, and Testing **36**
10.1 Verification Register . . . 36
10.2 Validation Register . . . 38
10.3 Example Test Cases . . . 40

11 Developer Requirements and Coding Standards **42**
11.1 Architecture Pattern: MVVM Across Two Modules . . . 42
11.2 Namespaces . . . 44
11.3 Class and Type Naming Conventions . . . 46
11.4 Code Style, Errors, and Concurrency . . . 48
11.5 Testing and Version Control . . . 49
11.6 UML Diagrams of the App's Structure . . . 49

12 Codes and Standards **53**
12.1 Process and Documentation Standards . . . 53
12.2 Product-Facing Standards and Guidelines . . . 54

II Appendices **56**

A Weekly Reports **57**
A.1 Week Report 9 (October 28, 2026) . . . 57
A.2 Week Report 8 (October 24, 2026) . . . 58
A.3 Week Report 7 (October 16, 2026) . . . 58
A.4 Week Report 6 (October 7, 2026) . . . 59
A.5 Week Report 5 (October 1, 2026) . . . 60
A.6 Week Report 4 (September 22, 2026) . . . 61
A.7 Week Report 3 (September 15, 2026) . . . 62

A.8 Week Report 2 (September 8, 2026) 62
A.9 Week Report 1 (September 1, 2026) 63

B AI Prompt Log **64**
B.1 How to Use This Appendix 64
B.2 Log Entries 64

Bibliography **68**

Glossary **70**

Acronyms **71**

Index **72**

List of Tables

1 Document Update History iv

5.1 Fall 2026: Biweekly Checkpoints 12
5.2 Spring 2027: Monthly Goals 12

9.1 User Stories to Requirements 31
9.2 Functional Requirements to Stories and Use Cases 32
9.3 Non-Functional Requirements to Stories and Use Cases 33
9.4 Constraints to Stories and Use Cases 33
9.5 Technical Requirements to Stories and Use Cases 34
9.6 Use Cases to Requirements 35

10.1 Verification Register 36
10.2 Validation Register 39
10.3 Example Test Cases 40

12.1 Process and Documentation Standards 53
12.2 Product-Facing Standards and Guidelines 54

List of Figures

8.1 Activity diagram for `UC-03` (Find Recipes from On-Hand Ingredients). Blue actions are performed by the user; green actions are performed by the system. When no recipe clears the minimum match threshold, the system falls back to showing the closest matches and what they are missing, rather than an empty result. 29

8.2 Activity diagram for `UC-11` (Add Pantry Items by Photo). If the system cannot confidently identify any ingredient in the photo, the user is routed to manual entry (`UC-02`) instead of being left with an empty or stuck screen. . 30

11.1 UML package diagram. ScrapApp depends on ScrapCore, never the reverse; only ScrapApp's Data adapters and ScrapCore's Networking folder touch external services. 50

11.2 UML class diagram of the ScrapCore domain model. Green classes are computed results produced by domain services rather than stored data. 51

11.3 UML class diagram of one MVVM feature slice (the Home search screen). The view model depends only on ScrapCore protocols and services; concrete repositories are injected by `AppDependencies`. 51

11.4 UML sequence diagram for an ingredient search (`UC-03`). The returned `ViewState` is `.loaded`, `.empty`, or `.failed`, so the view never shows a blank screen (REQ-D-05). 52

Part I

Project Definition and Plan

Chapter 1

Team Declaration

1.1 Team Name

Scrap

1.2 Team Members

- **Salome Orozco** is a fourth-year Industrial and Systems Engineering student at Stevens Institute of Technology, pursuing a minor in Finance. Originally from Texas, she came to Stevens to challenge herself and grow independently, choosing a course of study that blends the technical side of engineering with the strategic side of business. She currently serves as President of the First-Generation Student Union and thrives on hands-on projects that solve complex, structural problems.
- **Samah Keshiro** is a fourth-year Software Engineering major at Stevens Institute of Technology with interests in Product Management and AI. Serving on a club executive board revealed her strength for identifying requirements and managing timely delivery, and she is now working toward a career as a Technical Product Manager for software and digital products, building on AI engineering experience gained at Quest Diagnostics.
- **Deanna Gaber** is a fourth-year Software Engineering major at Stevens Institute of Technology focused on software development, systems design, and improving efficiency through technology-driven solutions. She values collaborative environments where ideas and responsibilities are shared and is using her coursework to strengthen her programming skills and prepare for a career in the technology industry.

1.3 Roles and Responsibilities

Because the team is developing Scrap largely with AI assistance, each member's role centers on the decisions and review that AI cannot make on its own, not just a slice of the codebase.

- **Deanna Gaber — Product & AI Integration Lead.** Owns the backlog and requirements: grooming user stories, deciding which features are "shall" versus "could" each sprint, running the team's weekly check-ins, and preparing the demo. Also sets the team's practices for using AI well — how to prompt, and when generated output needs a closer look before it is trusted — and keeps the AI Prompt Log accurate.
- **Samah Keshiro — iOS App & Backend Lead.** Owns the technical build end to end: the Swift and SwiftUI app (its MVVM structure, screens, and offline caching), the Supabase schema and client integration, the Spoonacular API integration and its failure handling, and getting builds into TestFlight. Also coordinates XCTest/XCUITest coverage and triages bugs reported through GitHub Issues.
- **Salome Orozco — Business Strategy & Marketing Lead.** Focuses on the business side of Scrap: understanding what makes it different from established recipe and meal-planning apps and making sure that positioning carries through the product. Leads the companion website as a marketing and advertising vehicle — messaging, content, and how the team introduces Scrap to new users — and, drawing on her Finance minor, keeps an eye on the team's free-tier budget (Supabase storage, the daily Spoonacular API cap) so it doesn't become a surprise.

Regardless of these leads, no team member merges AI-generated code without a second member reviewing it first; AI moves fast but also introduces subtle bugs that are easy to miss without a second pair of eyes.

1.4 Project Proposal

Our team, Scrap, proposes to develop a recipe and meal-planning application designed to help users make better use of the food they already have while reducing unnecessary food waste. The application will allow users to enter ingredients they currently have available and receive recipe recommendations based on those ingredients. Rather than requiring users to shop for an entirely new set of ingredients for every recipe, the system will prioritize meals that make the greatest use of what is already in the user's kitchen.

The application will be centered around a simple search experience where users can search for ingredients and apply additional filters to personalize their results. Recipes will be ranked according to the percentage of ingredients the user already has, allowing users to quickly identify meals that require the fewest additional purchases. When additional ingredients are needed, users will be able to compile those items into a grocery list. The system will also allow users to contribute new recipes, creating the potential for the application to become a more social and collaborative platform over time.

Chapter 2

Mission Statement

The mission of Scrap is to make cooking easy, affordable, fun, and less wasteful by helping people decide what to cook using the ingredients they already have. Scrap connects users with recipes that fit their available ingredients, budget, and preferences, and builds a social platform where users can share their own recipes, discover recipes from other home cooks, and feel connected to a broader community of people who enjoy cooking.

Chapter 3

Key Drivers

Scrap is driven by four core values: making cooking easy, affordable, fun, and less wasteful. The drivers described below explain where these values come from and why each one matters to our team.

The primary motivation for Scrap comes from our experiences as college students transitioning into apartments and becoming responsible for cooking and grocery shopping on our own. Cooking regularly can be challenging when balancing classes, work, social responsibilities, and other commitments. For students who are still developing their cooking skills, deciding what to make can become an additional source of stress, particularly when there are only a limited number of ingredients available.

Budget is another major driver behind the proposed solution. College students often have limited disposable income and need to be thoughtful about how they spend money on groceries. Many existing recipes assume that users have access to a large variety of ingredients or require purchasing several items that may only be used once. This can make following a recipe more expensive than expected and can leave users with partially used ingredients that eventually expire. For students living alone or with a small number of roommates, this problem is especially noticeable because purchasing larger quantities of food does not always make sense for smaller portions.

Another challenge we identified is the difficulty of finding recipes that accommodate what a person already has in their kitchen. Traditional recipe searches typically begin with a desired meal, causing users to search for a recipe first and then purchase the ingredients required to make it. Scrap approaches this problem from the opposite direction: users begin with what they already have. By entering their available ingredients, users can discover recipes that make use of those items and determine which additional ingredients, if any, they need to purchase.

Food waste is also an important motivation for the project. Food waste is a large and costly problem in the United States: over a third of the food produced is never eaten [1], and a large share of that waste happens at the household level, driven by over-purchasing, poor planning, and forgotten ingredients [2]. The cost to individual households is significant as well, and using up leftovers before they spoil is one of the most effective ways for a household to cut that cost down [3]. Ingredients are frequently purchased with good intentions but are forgotten about or left unused until they expire. Scrap is intended to encourage users to

think about their existing food inventory before purchasing additional groceries. Ranking recipes based on the percentage of ingredients a user already has can help users prioritize meals that use ingredients that might otherwise go to waste. In this way, the application connects personal benefits, such as saving money and reducing grocery trips, with a broader goal of reducing unnecessary food waste.

Finally, the social and enjoyable side of cooking is a key driver behind Scrap. We want the application to be more than a basic recipe search engine; we envision it as a social, community-driven platform that makes deciding what to cook fun rather than a chore. Users will be able to search for recipes, explore recommendations, create grocery lists, and contribute their own recipes to the system, as well as browse and draw inspiration from recipes shared by other users. This social, user-driven approach reflects our value of making cooking fun: it gives users a sense of participation and connection to a community of fellow home cooks, while also allowing the recipe collection to grow organically over time.

Chapter 4

Key Constraints

One of the major constraints for Scrap is the existing competition in the food and recipe application market. There are already many established applications and websites that provide recipe searches, meal planning, grocery lists, ingredient-based searches, and other related features. Because of this, simply creating another recipe application may not provide enough differentiation. Our project will need to clearly define what makes Scrap useful and distinct, particularly its focus on maximizing ingredients that users already have and reducing food waste. The interface and functionality will need to support this central idea rather than simply reproducing features that are already common in existing applications.

Another significant constraint is our team's development experience. While all three team members have experience developing websites and working with software development tools, we have not previously built a complete application of this type. Developing Scrap will therefore require us to learn and apply new technologies and development practices — principally Swift, SwiftUI, and Supabase, the stack the team has chosen for this project — along with other unfamiliar tools as they come up. We will need to account for this learning curve when determining the scope of the project and deciding which features are realistic to implement within the available time.

The project is also constrained by the need to balance functionality with development complexity. Our initial vision includes ingredient-based recipe searches, filters, recipe rankings, grocery lists, and user-submitted recipes. Although these features would make the application more useful, implementing all of them successfully may require significant time and technical effort. We will need to prioritize the core functionality of the application and distinguish between essential features for the minimum viable product and features that could be added if time permits.

Data management presents another potential constraint. The application depends on a collection of recipes and their associated ingredients in order to provide meaningful recommendations. Rather than building a recipe database from scratch, the team has decided to source recipe and ingredient data from the Spoonacular API, merged with recipes users contribute directly, and to store it in a relational Supabase (Postgres) schema (see Requirements). The system will still need to account for differences in how an ingredient is named, measured, or spelled between a user's pantry and a recipe's ingredient list, so that recipe recommendations stay accurate.

Finally, usability is an important constraint because the application is intended to be simple and convenient for users who may already be short on time. The core experience should allow users to quickly enter what they have and receive useful recommendations without requiring extensive setup. As we develop the application, we will need to balance the number of available filters and features with the simplicity of the interface. The final product should demonstrate the core concept of Scrap clearly while remaining achievable within the technical and time limitations of the project.

4.1 Tradeoffs and a Documented Design Iteration

Each constraint above forced a specific tradeoff rather than staying abstract. The budget constraint means grocery costs are shown as estimates rather than real-time prices, since no free, real-time grocery price source is available (REQ-C-05). The team's inexperience with the chosen stack means the project's scope had to assume a learning curve rather than senior-level velocity from day one (see Risks and Assumptions, below). The functionality-versus-complexity constraint produced the *shall*/*could* split in Requirements itself: recipe video, leftover tracking, and expiration alerts were marked as bonus rather than required so the minimum viable product stays achievable (REQ-F-07, REQ-F-13, REQ-F-14). The data-management constraint means the app must tolerate an imperfect third-party data source rather than assuming clean, consistent ingredient data (REQ-T-11). The usability constraint means filters are deliberately limited to time, difficulty, and equipment rather than exposing every attribute Spoonacular returns.

The team has also closed one complete design-iteration loop so far, and is documenting it explicitly here rather than leaving it implied by version history alone. After the team's Milestone 1 class presentation, reviewers raised usability feedback that two steps in the app's core loop were still manual and tedious: typing every pantry item in by hand, and tracking each item's expiration date without any reminder. Rather than deferring that feedback to a later milestone, the team closed the loop directly in this report:

1. **Feedback:** Milestone 1 presentation reviewers identified manual pantry entry and untracked expiration as the two biggest friction points in the proposed design.

2. **Design change:** the team added two new bonus functional requirements the same week — REQ-F-19 (photo-based pantry entry) and REQ-F-20 (automatic shelf-life estimation and expiration reminders) — each with its own use case (UC-11, UC-12), traced through Requirements and the Traceability Matrix, and recorded in Document History (v1.22).

3. **Planned recheck:** each new requirement has its own verification row in the Verification Register — VER-005 for REQ-F-19, VER-006 for REQ-F-20 (see Verification, Validation, and Testing) — so the team has a concrete, planned way to confirm the change actually resolved the feedback once Scrap is built. Neither check has run yet, since implementation has not started; both are labeled **PENDING**rather than treated as already resolved.

Chapter 5

Development Plan

This chapter summarizes the technical plan the team is building Scrap against: the mobile application platform, the backend and data sources it depends on, the companion website, and how the team plans to develop and test the app. The specific requirements this plan is built from are detailed in Chapter 7; this chapter describes the plan as a whole and the reasoning behind it.

5.1 Mobile Application

Scrap will be built natively for iOS using Swift and SwiftUI [4] (REQ-T-01), rather than a cross-platform framework, so the team can build directly against Apple's current APIs and design guidelines (REQ-T-05) without the overhead of a cross-platform abstraction layer. The app will follow an MVVM architecture that keeps ingredient-matching, ranking, and grocery-list logic separate from the SwiftUI view layer (REQ-T-02), and will cache pantry contents and previously loaded recipes locally so core functionality keeps working offline (REQ-T-03). A cross-platform framework such as Flutter, which would allow the team to also support Android, remains a possible future addition once the core iOS app is complete, but is out of scope for this milestone.

5.2 Backend

Scrap will use Supabase, a hosted Postgres database with built-in authentication and file storage, as its backend (REQ-T-08), rather than the team building and hosting its own server. This lets the team focus development time on the app itself instead of backend infrastructure, while still getting a relational database, user accounts, and storage for user-uploaded photos and videos. The Supabase schema will include relational tables for users, pantry items, recipes, recipe ingredients, and grocery list items (REQ-T-09), and the app will reach Supabase through its official Swift SDK over HTTPS [5] (REQ-T-04).

5.3 Recipe and Ingredient Data

Rather than building a recipe database from scratch, Scrap will source recipe and ingredient data from the Spoonacular API [6], including its ingredient-based search endpoint, merged with recipes users contribute directly (REQ-T-10). The app will be built to handle an unexpected Spoonacular response — missing fields, a rate-limit error, or no results — without crashing or showing a blank screen (REQ-T-11), since the app depends on a third-party service it does not control. As a bonus feature beyond the minimum viable product, the team may also add Open Food Facts as a free product-lookup source so users can scan a barcode to add an item to their pantry (REQ-F-18).

5.4 Companion Website

In addition to the iOS app, the team will build and host a companion marketing website separately from the app — a static site on Vercel or Netlify, sharing no codebase or backend with the app (REQ-T-13). The website is a separate product intended to introduce Scrap to visitors rather than a part of the app's functionality, so it has no corresponding use case in Chapter 8.

5.5 Workspace

We will use GitHub to keep our code in one place and work on the app together. Each person will have their own branch so we can work on different parts of the app without changing the main version before the work is ready. Once a feature is working, we can merge it into the main branch.

We will also use shared documents for our report, research, and other project materials. GitHub Issues can be used to keep track of things that need to be fixed or completed (REQ-C-06).

For the actual app, we plan to use Xcode, Swift, and SwiftUI (REQ-T-01). Supabase will be used for the backend (REQ-T-08), and Spoonacular will provide the recipe and ingredient information (REQ-T-10).

5.6 Communication Plan

We will check in with each other throughout the week instead of waiting until our next class or meeting to talk about the project. These check-ins will give us a chance to say what we have finished, what we are currently working on, and if we are stuck on anything.

For quick questions and updates, we will use our group messaging platform. We will use GitHub for code, tasks (kanban board), and bugs, and shared documents for our report and research.

If someone is falling behind or runs into a problem that affects another part of the project, we will let the group know as soon as possible. This will help us make changes early instead

of finding out about a problem right before a deadline.

5.7 Risks and Assumptions

The team is assuming that Supabase's and Spoonacular's free tiers will remain available and sufficient for the scope of this project (REQ-C-01); if either service changes its free-tier limits or pricing, the team would need to reduce usage, find a substitute, or revisit this constraint. The team is also assuming that the Spoonacular API's recipe and ingredient data is accurate and complete enough to support ingredient matching without extensive manual correction (REQ-T-10), and that the two most recent major iOS releases cover enough of the team's own devices and any beta testers to develop and test against (REQ-T-01).

The team's biggest risk is unfamiliarity with the chosen stack: all three members are learning Swift, SwiftUI, and Supabase at the same time, which could slow early development more than a team already experienced with these tools (see Week Report 4 in Chapter A). The team's contingency is to start with a small, well-scoped first feature and check in regularly so that one person's blockers do not stall the whole team, rather than discovering the problem only near a deadline. A second risk is dependence on the Spoonacular API as a third party the team does not control; an outage, a changed response format, or a failed request could make the app behave incorrectly unless it is built and tested against those failure cases ahead of time (REQ-T-11). A third risk is scope: the team's initial feature list is broader than what can realistically be finished in the time available, which is why several features — recipe video, leftover tracking, expiration alerts, and barcode entry — have already been marked as bonus rather than required (REQ-F-07, REQ-F-13, REQ-F-14, REQ-F-18). A fourth risk is tooling access: SwiftUI (REQ-T-01) only builds, previews, and runs inside Xcode, and Xcode only runs on macOS, but none of the three team members owns a Mac — the team's only Mac access is through Stevens library computers, which are shared with other students and not always available. Treating Xcode as necessary for all day-to-day development would make that shared, limited resource a constant bottleneck. The team's mitigation is to split the codebase into two pieces: a platform-independent Swift package (ScrapCore) holding the data models, the recipe-matching and ranking logic, serving-size scaling, grocery list generation, and the networking layer, which the official Swift toolchain builds and tests on any team member's own Windows machine; and a thin SwiftUI layer (ScrapApp) that depends only on ScrapCore's own interfaces, which is the only part that actually requires a Mac. This confines library computer time to periodic batched sessions — wiring already-written SwiftUI files into an Xcode project, previewing screens, and running the simulator — rather than continuous use, while the team's day-to-day logic development and testing can happen on any machine, at any time.

5.8 Timeline and Milestones

The team is planning the fall semester in two-week increments, building toward a complete working prototype by early December. Spring semester goals are further out and broader, so

they are planned one month at a time instead. Where a checkpoint below is also an official 423-IDE401/424-IDE402 course milestone, it is labeled as such.

Table 5.1: Fall 2026: Biweekly Checkpoints

Date	Goal
September 24, 2026	**Milestone 1: Design Proposal.** This report is the team's Milestone 1 submission (Version 1.31), presented to the class this week, covering the mission, key drivers and constraints, development plan, user stories, requirements, use cases, and traceability matrix described above.
October 8, 2026	The Xcode project is set up and connected to the Supabase backend (REQ-T-08); the first app screens are in progress.
October 22, 2026	The team's first end-to-end slice is working: pantry entry connected to a ranked Spoonacular search result.
November 5, 2026	The remaining core screens (grocery list generation, recipe details) are built, and the Supabase schema (REQ-T-09) is fully wired up.
November 19, 2026	**Milestone 2: Preliminary Design Review.** The design, finalized concept evaluation, and analysis are documented for review.
December 3, 2026	**Working prototype complete.** Pantry entry, ingredient-based recipe search and ranking, and grocery list generation all function together, ahead of winter break.

Table 5.2: Spring 2027: Monthly Goals

Month	Goal
January 2027	The team recruits a small group of beta users and begins testing through Apple TestFlight (REQ-T-12).
February 2027	**Milestone 3: Performance and Cost Review with Alpha Prototype Demonstration.** An alpha version with every core functional requirement (Chapter 7) is complete, and the team iterates on the feedback gathered from January's beta testing.
March 2027	**Milestone 4: Optimized Design Review with Beta Prototype Demonstration.** Beta testing and iteration continue, and the team begins growing an initial user base ahead of a public launch.
April 2027	The team continues iterating on the product and growing the user base, and prepares the final report and demo.
May 2027	**Milestone 5: Submission of the Final Report.** The final report is due, including test evidence, the completed companion website (REQ-T-13), and the results of the team's beta testing and user-growth efforts.

5.9 Testing Plan

Scrap will be tested through a mix of manual testing and automated checks. Ingredient-matching, ranking, and grocery-list logic, including the Supabase queries behind them, will be covered by XCTest unit tests, and core flows such as account creation and recipe search will be covered by XCUITest UI tests (REQ-T-06). The team will also manually check the app's integration with the Spoonacular API. Before any public release, the app will be distributed to a small group of test users through Apple TestFlight to gather feedback ahead of a general launch (REQ-T-12), and defects found during testing will be tracked using GitHub Issues in the project's existing repository (REQ-C-06). The specific verification methods, validation scenarios, and example test cases that will exercise this plan against Scrap's requirements are detailed in Chapter 10.

Chapter 6

User Stories

The stories below fall into two groups. End-user stories (US-01 through US-08) come from the team's first round of stakeholder interviews, conducted during Week 3 (see Weekly Reports); all four interviewees are college seniors, Scrap's primary target user (see Mission Statement and Key Drivers). Other stakeholder stories (US-09 through US-14) were derived by the team to represent perspectives beyond the end user: the developers who build and maintain Scrap, the business stakeholders responsible for its long-term viability, and the moderators who keep its community feed safe. Each story is written in the standard "As a _____, I want _____, so that _____" form and is mapped to the requirements it produced in Requirements. The complete story–requirement–use case mapping is in Traceability Matrix.

6.1 Interview Summary

Interviewee 1. *College senior, on-campus apartment.* Cooks 1–2 meals/day; finds recipes on Instagram; hardest part is the time between finding a recipe and getting the ingredients; limited space leads to batch cooking and consolidating appliances; relies on friends for appliances; about 5–10% food waste; sees cooking as highly social. Stories: US-01, US-02.

Interviewee 2. *College senior, five-person apartment.* Cooks about 2 meals/day; finds recipes on TikTok/Instagram and wants written steps alongside video; hardest part is motivation versus eating out; finishes leftovers about half the time; about 25% food waste, improved by frozen vegetables; uses friends' cooking as inspiration. Stories: US-03, US-04.

Interviewee 3. *College senior, living at home.* Cooks about twice a week; chooses meals from ingredients already on hand to save money; budget heavily affects choices; has had ingredients expire; dislikes social-media recipes that hide the full recipe; wants protein, prep time, difficulty, and substitution suggestions. Stories: US-05, US-06.

Interviewee 4. *College senior, apartment.* Cooks about 14 times/week; shops for specific meals; struggles with portioning and grocery cost; sometimes buys groceries already on hand; searches Google and checks ingredients by hand; likes video, dislikes buried ingredient lists;

very often stuck with a random ingredient; about 10% food waste from spoilage. Stories: US-07, US-08.

6.2 End-User Stories

US-01: Closing the gap between finding a recipe and cooking it As a college student cooking in a small kitchen, I want to see which recipes I can already make, so that I don't waste time gathering ingredients.
Source: Interviewee 1. **Maps to:** REQ-F-03, REQ-F-04, REQ-F-09, REQ-F-11, REQ-F-12.

US-02: Cooking as a social activity As a college student, I want to share what I'm cooking with friends, so that cooking feels social instead of solitary.
Source: Interviewee 1. **Maps to:** REQ-F-01, REQ-F-15, REQ-F-16.

US-03: Video inspiration with written instructions As a college student who finds recipes on social media, I want written steps alongside any video, so that I can actually follow the recipe.
Source: Interviewee 2. **Maps to:** REQ-F-04, REQ-F-05, REQ-F-06, REQ-F-07, REQ-NF-01.

US-04: Using up leftovers and getting inspiration from friends As a college student who often has leftovers, I want recipe ideas that use them up, so that less of my food goes to waste.
Source: Interviewee 2. **Maps to:** REQ-F-02, REQ-F-12, REQ-F-13, REQ-F-16.

US-05: Cooking from what is already at home, on a budget As a budget-conscious college student, I want recipes recommended from what I already have, so that I spend less and waste less.
Source: Interviewee 3. **Maps to:** REQ-F-02, REQ-F-03, REQ-F-06, REQ-F-08, REQ-F-10, REQ-F-14.

US-06: The whole recipe up front As a college student browsing recipes on my phone, I want the full recipe on one screen, so that I don't have to hunt for it elsewhere.
Source: Interviewee 3. **Maps to:** REQ-F-05, REQ-F-07, REQ-NF-01, REQ-NF-03.

US-07: Portions, budget, and not buying what is already at home As a budget-conscious college student, I want a grocery list limited to what I actually lack, so that I don't overspend or overbuy.
Source: Interviewee 4. **Maps to:** REQ-F-02, REQ-F-09, REQ-F-10, REQ-F-12, REQ-F-13.

US-08: Stop manually matching ingredients; stop letting food go bad As a college student, I want the app to match my pantry to recipes automatically, so that I stop doing it by hand.
Source: Interviewee 4. **Maps to:** REQ-F-03, REQ-F-05, REQ-F-07, REQ-F-14.

6.3 Other Stakeholder Stories

The stories below represent stakeholders beyond the end user: the developers building Scrap, the business/product side responsible for its long-term viability, and the moderators who keep the community feed safe. They were derived by the team rather than gathered from an interview, and they directly informed the iOS-specific technical requirements in Requirements.

US-09: Isolated business logic As a developer, I want the matching and grocery-list logic reachable through a documented API, so that I can change it without breaking the UI.
Source: Derived by the team (developer stakeholder). **Maps to:** REQ-T-02, REQ-T-04.

US-10: Confidence to change core logic As a developer, I want automated tests on the app's core logic, so that changes don't silently break it.
Source: Derived by the team (developer stakeholder). **Maps to:** REQ-T-06.

US-11: An app that works without a connection As a developer, I want pantry data stored locally on the device, so that the app works offline.
Source: Derived by the team (developer stakeholder). **Maps to:** REQ-T-03.

US-12: A native, approvable app As a business stakeholder, I want Scrap built to Apple's guidelines, so that it feels native and passes App Store review.
Source: Derived by the team (business stakeholder). **Maps to:** REQ-T-01, REQ-T-05.

US-13: Bringing users back As a business stakeholder, I want timely push notifications, so that users keep coming back to the app.
Source: Derived by the team (business stakeholder). **Maps to:** REQ-T-07, REQ-F-14.

US-14: Keeping the community feed safe As a community moderator, I want to remove reported content, so that the feed stays safe.
Source: Derived by the team (community stakeholder). **Maps to:** REQ-F-17.

Chapter 7

Requirements

This chapter presents Scrap's initial requirements, divided into functional requirements (REQ-F), non-functional requirements (REQ-NF), constraints (REQ-C), and technical requirements for the iOS platform (REQ-T). Each requirement lists the user stories it came from (User Stories) and the use cases that exercise it (Use Cases). Requirements that do not come directly from an interview are marked as derived and name their basis. Constraints are drawn from the Key Constraints chapter. The requirements support the four core values in the Mission Statement: making cooking easy, affordable, fun, and less wasteful.

7.1 Functional Requirements

REQ-F-01: User accounts. The system shall allow a user to create an account, log in, and log out, so that pantry data, contributed recipes, and posts are tied to that user.
Source: US-02. **Use cases:** UC-01, UC-09, UC-10.

REQ-F-02: Pantry inventory. The system shall allow a user to add, edit, and remove ingredients they have on hand (including frozen items and saved leftovers), with an optional quantity for each.
Source: US-04, US-05, US-07. **Use cases:** UC-02, UC-05, UC-07.

REQ-F-03: Ingredient-match search. The system shall recommend recipes based on the user's pantry and rank them by the percentage of each recipe's ingredients the user already has.
Source: US-01, US-05, US-08. **Use cases:** UC-03, UC-08.

REQ-F-04: Recipe filters. The system shall let the user filter and sort recommended recipes by total time, difficulty, and required kitchen equipment.
Source: US-01, US-03. **Use cases:** UC-03.

REQ-F-05: Complete recipe view. The system shall display a recipe's full ingredient list and complete written step-by-step instructions together on one page inside the app,

without redirecting the user to an external site.
Source: US-03, US-06, US-08. **Use cases:** UC-04.

REQ-F-06: Recipe facts. The system shall display, for each recipe, the prep and cook time, a difficulty rating, and nutrition facts including protein per serving.
Source: US-03, US-05. **Use cases:** UC-04.

REQ-F-07: Recipe video. The system could allow a recipe to include a video, shown alongside (never in place of) the written instructions. This is a bonus feature, not required for the minimum viable product.
Source: US-03, US-06, US-08. **Use cases:** UC-04, UC-09.

REQ-F-08: Ingredient substitutions. The system shall suggest substitutions for ingredients the user is missing, preferring substitutes already in the user's pantry.
Source: US-05. **Use cases:** UC-05.

REQ-F-09: Grocery list. The system shall generate a grocery list containing only the ingredients a selected recipe needs that are not already in the user's pantry.
Source: US-01, US-07. **Use cases:** UC-06.

REQ-F-10: Estimated cost. The system shall display an estimated cost for each recipe and for each generated grocery list.
Source: US-05, US-07. **Use cases:** UC-06.

REQ-F-11: Equipment profile. The system shall allow the user to record which kitchen appliances and tools they own, and use this profile when filtering recipes.
Source: US-01. **Use cases:** UC-01, UC-03.

REQ-F-12: Serving scaling. The system shall allow the user to change a recipe's number of servings and automatically rescale all ingredient quantities.
Source: US-01, US-04, US-07. **Use cases:** UC-07.

REQ-F-13: Leftover tracking. The system could allow the user to log leftover portions from a cooked recipe to their pantry and could suggest recipes that reuse those leftovers. This is a bonus feature, not required for the minimum viable product.
Source: US-04, US-07. **Use cases:** UC-07.

REQ-F-14: Expiration alerts. The system could allow the user to record an expiration date for pantry items, notify the user when an item is close to expiring, and prioritize recipes that use soon-to-expire items. This is a bonus feature, not required for the minimum viable product.
Source: US-05, US-08. **Use cases:** UC-02, UC-08.

REQ-F-15: Recipe contribution. The system shall allow a logged-in user to submit a recipe (ingredients, written steps, servings, time, difficulty, and optional photo or video) for other users to discover.
Source: US-02. **Use cases:** UC-09.

REQ-F-16: Community feed. The system shall allow a logged-in user to post photos of dishes they have cooked, link the post to a recipe, and browse a feed of posts from other users.
Source: US-02, US-04. **Use cases:** UC-10.

REQ-F-17: Content moderation. The system shall allow a user to report a post or contributed recipe, and allow an administrator to remove reported content that violates community guidelines.
Source: US-14. **Use cases:** UC-10.

REQ-F-18: Barcode pantry entry. The system could allow a user to scan a product barcode to add it to their pantry, using a free product-lookup source such as Open Food Facts. This is a bonus feature, not required for the minimum viable product.
Source: Derived by the team's technical plan. **Use cases:** UC-02.

REQ-F-19: Photo-based pantry entry. The system could allow a user to add pantry items by taking or uploading a photo of their ingredients, using image recognition to identify them, as a faster alternative to typing each item in manually. This is a bonus feature, not required for the minimum viable product.
Source: Derived from usability feedback the team received after presenting Milestone 1. **Use cases:** UC-02, UC-11.

REQ-F-20: Automatic shelf-life estimation and expiration reminders. The system could estimate how long a fresh, unpackaged ingredient (such as a vegetable) will likely last based on its type, without requiring the user to enter an expiration date manually, and could notify the user with recipe suggestions that use that ingredient as it nears the end of its estimated shelf life. This is a bonus feature, not required for the minimum viable product.
Source: Derived from usability feedback the team received after presenting Milestone 1. **Use cases:** UC-02, UC-08, UC-12.

7.2 Non-Functional Requirements

REQ-NF-01: Usability. A user shall be able to reach a recipe's complete ingredient list and instructions within one tap of selecting it from the results list.
Source: US-03, US-06. **Use cases:** UC-03, UC-04.

REQ-NF-02: Performance. Ranked recipe results shall be returned within 2 seconds for a pantry of up to 100 items.
Source: Derived by the team from US-05 and US-08 (the core ingredient-matching workflow must feel instant). **Use cases:** UC-03, UC-08.

REQ-NF-03: Mobile responsiveness. All pages shall be fully usable on a phone-sized screen (360 px wide or larger), since users cook and shop with their phones in hand.
Source: US-06. **Use cases:** UC-02, UC-04, UC-06.

REQ-NF-04: Security and privacy. User passwords shall be stored only as salted hashes, and users shall control whether their posts and contributed recipes are public.
Source: Derived by the team from US-02 (sharing requires accounts and user trust). **Use cases:** UC-01, UC-10.

REQ-NF-05: Accessibility. The interface shall meet WCAG 2.1 Level AA guidelines [7], and every recipe video shall be accompanied by complete written steps.
Source: Derived by the team from US-03 and US-06. **Use cases:** UC-04, UC-09. See the Codes and Standards chapter for how this standard became a requirement and how it will be checked.

7.3 Constraints

REQ-C-01: Budget. The system shall be built and hosted using free-tier services — Supabase for the backend and storage, Spoonacular for recipe data, and Vercel or Netlify for the companion website — which limits storage for user-uploaded photos and videos and caps the number of Spoonacular API calls available per day.
Source: Key Constraints. **Use cases:** UC-03, UC-09, UC-10.

REQ-C-02: Schedule. The system shall be delivered on the senior-design milestone schedule within the current academic year.
Source: Key Constraints. **Use cases:** All (UC-01–UC-10).

REQ-C-03: Team size. The system shall be designed, built, and tested by a three-person student team.
Source: Key Constraints. **Use cases:** All (UC-01–UC-10).

REQ-C-04: Data sources. Recipe and nutrition data shall come from the Spoonacular API (free tier), optionally supplemented by Open Food Facts for product lookups; user-contributed recipe content shall be original or properly licensed and shall not be copied from social media or other sites.
Source: Key Constraints. **Use cases:** UC-04, UC-09.

REQ-C-05: Price data. Grocery costs shall be shown as estimates, because no free, real-time grocery price source is assumed to be available.
Source: Key Constraints. **Use cases:** UC-06.

REQ-C-06: Bug tracking. The team shall track defects found during testing using GitHub Issues in the project's existing repository, so testing feedback is not lost between weekly reports.
Source: Derived by the team's technical plan. **Use cases:** All (UC-01–UC-10).

7.4 Technical Requirements

These requirements capture how Scrap will be built, based on the team's technical plan: a native iOS app backed by Supabase, recipe data sourced from Spoonacular, and a companion website kept separate from the app. Requirements without a specific user-story source were derived directly from that plan.

REQ-T-01: Native platform. The app shall be built natively in Swift and SwiftUI, targeting the two most recent major iOS releases. A cross-platform framework such as Flutter for Android support remains a possible future addition and is out of scope for this milestone.
Source: US-12. **Use cases:** All (UC-01–UC-10).

REQ-T-02: MVVM architecture. The app shall follow an MVVM architecture that keeps ingredient-matching, ranking, and grocery-list logic in view models separate from the SwiftUI view layer.
Source: US-09. **Use cases:** UC-03, UC-06.

REQ-T-03: Offline-first persistence. Pantry contents and previously loaded recipes shall be cached locally (SwiftData or Core Data) on top of the Supabase backend, so pantry management and recipe browsing keep working without a network connection.
Source: US-11. **Use cases:** UC-02, UC-03.

REQ-T-04: Networking. The app shall reach Supabase through its official Swift SDK and reach Spoonacular through its REST API, with every request made over HTTPS.
Source: US-09. **Use cases:** All (UC-01–UC-10).

REQ-T-05: Platform guidelines. The app shall follow Apple's Human Interface Guidelines [8] (navigation, typography, and Dynamic Type support) and Apple's App Store Review Guidelines [9].
Source: US-12. **Use cases:** All (UC-01–UC-10). See the Codes and Standards chapter for how these guidelines became a requirement and how they will be checked.

REQ-T-06: Automated testing. The ingredient-matching and grocery-list logic, including the Supabase queries behind them, shall be covered by XCTest unit tests, and account creation and recipe search shall be covered by XCUITest.
Source: US-10. **Use cases:** UC-01, UC-03, UC-06.

REQ-T-07: Push notifications. Expiration alerts shall be delivered through Apple Push Notification service (APNs), requested via the standard iOS notification permission prompt.
Source: US-13. **Use cases:** UC-08.

REQ-T-08: Backend platform. The app shall use Supabase (hosted Postgres, authentication, and file storage) as its backend, so the team is not building or hosting its own server infrastructure.
Source: Derived by the team's technical plan. **Use cases:** All (UC-01–UC-10).

REQ-T-09: Database schema. The Supabase schema shall include relational tables for users, pantry items, recipes, recipe ingredients, and grocery list items.
Source: Derived by the team's technical plan. **Use cases:** UC-02, UC-03, UC-06.

REQ-T-10: Recipe and ingredient data source. Recipe and ingredient data shall come from the Spoonacular API, including its ingredient-based search endpoint, merged with recipes users contribute directly through Scrap.
Source: Derived by the team's technical plan. **Use cases:** UC-03, UC-04, UC-08, UC-09.

REQ-T-11: Third-party API resilience. The app shall handle an unexpected Spoonacular response — missing fields, a rate-limit error, or no results — without crashing or showing a blank screen.
Source: Derived by the team's technical plan. **Use cases:** UC-03, UC-04.

REQ-T-12: Beta distribution. The app shall be distributed to a small group of test users through Apple TestFlight before any public App Store release, to gather feedback ahead of a general launch.
Source: Derived by the team's technical plan. **Use cases:** All (UC-01–UC-10).

REQ-T-13: Companion website. A companion marketing website shall be built and hosted separately from the app (a static site on Vercel or Netlify), sharing no codebase or backend with the iOS app.
Source: Derived by the team's technical plan. **Use cases:** None — a separate product from the app (see the Traceability Matrix).

Chapter 8

Use Cases

The use cases below describe how users interact with Scrap. They are based on the User Stories and on the team's starting list of use cases from Week 3 (see Weekly Reports). The primary actor in every use case is a *Scrap user* (a home cook, typically a college student). Every use case maps to at least one functional requirement in Requirements. Requirements that apply broadly rather than to a specific use case — REQ-C-02, REQ-C-03, REQ-C-06, REQ-T-01, REQ-T-04, REQ-T-05, REQ-T-08, and REQ-T-12 — are not repeated below; the complete mapping is in the Traceability Matrix.

UC-01: Create Account and Kitchen Profile
Requirements: REQ-F-01, REQ-F-11, REQ-NF-04, REQ-T-06.
Actor: Scrap user.
Preconditions: The user has opened Scrap and is not logged in.
Main flow:

1. The user selects *Sign up* and enters an email and password.
2. The system validates the input, stores the password as a salted hash, and creates the account.
3. The system prompts the user to select the kitchen appliances and tools they own.
4. The system saves the equipment profile and opens the pantry screen.

Alternate flows: 2a. The email is already registered: the system offers to log in instead. 3a. The user skips the equipment step: the system assumes no equipment filter until the profile is completed.
Postconditions: The user is logged in with an account and a saved equipment profile.

UC-02: Manage Pantry
Requirements: REQ-F-02, REQ-F-14, REQ-F-18, REQ-F-19, REQ-F-20, REQ-NF-03, REQ-T-03, REQ-T-09.
Actor: Scrap user.
Preconditions: The user is logged in.
Main flow:

1. The user opens the pantry.
2. The user adds an ingredient, optionally with a quantity and an expiration date.
3. The user edits or removes existing items as they are used up.
4. The system saves the pantry and updates any expiration reminders.

Alternate flows: 2a. The ingredient is not recognized: the system suggests the closest matching ingredient names.
Postconditions: The pantry reflects what the user has on hand.

UC-03: Find Recipes from On-Hand Ingredients
Requirements: REQ-F-03, REQ-F-04, REQ-F-11, REQ-NF-01, REQ-NF-02, REQ-C-01, REQ-T-02, REQ-T-03, REQ-T-06, REQ-T-09, REQ-T-10, REQ-T-11.
Actor: Scrap user.
Preconditions: The user is logged in and has at least one pantry item.
Main flow:

1. The user selects *What can I make?*
2. The system matches the pantry against the recipe catalog and ranks recipes by the percentage of ingredients on hand.
3. The user optionally filters by total time, difficulty, or owned equipment.
4. The user selects a recipe, continuing to UC-04.

Alternate flows: 2a. No recipe meets a minimum match: the system shows the closest matches and their missing ingredients.
Postconditions: The user sees a ranked, filtered list of recipes.

UC-04: View Recipe Details
Requirements: REQ-F-05, REQ-F-06, REQ-F-07, REQ-NF-01, REQ-NF-03, REQ-NF-05, REQ-C-04, REQ-T-10, REQ-T-11.
Actor: Scrap user.
Preconditions: The user has selected a recipe.
Main flow:

1. The system displays the full ingredient list, marking which items are in the pantry.
2. The system displays the complete written steps, the recipe facts (time, difficulty, protein and nutrition), and the video if one exists.
3. The user may continue to UC-05, UC-06, or UC-07.

Alternate flows: 2a. The recipe has no video: the system shows the written steps only.
Postconditions: The user has everything needed to cook the recipe on one screen.

UC-05: Substitute a Missing Ingredient
Requirements: REQ-F-02, REQ-F-08.

Actor: Scrap user.
Preconditions: The user is viewing a recipe with at least one missing ingredient.
Main flow:

1. The user selects a missing ingredient.
2. The system lists substitutes, placing those already in the pantry first.
3. The user selects a substitute, and the recipe view marks the ingredient as covered.

Alternate flows: 2a. No known substitute exists: the system offers to add the ingredient to the grocery list (UC-06).
Postconditions: The missing ingredient is covered by a substitute or queued for purchase.

UC-06: Generate Grocery List
Requirements: REQ-F-09, REQ-F-10, REQ-NF-03, REQ-C-05, REQ-T-02, REQ-T-06, REQ-T-09.
Actor: Scrap user.
Preconditions: The user is viewing a recipe with at least one missing ingredient.
Main flow:

1. The user selects *Add missing to grocery list*.
2. The system adds only the ingredients not already in the pantry, scaled to the chosen number of servings.
3. The system displays the grocery list with an estimated cost for each item and in total.
4. The user checks off items while shopping; checked items are moved into the pantry.

Alternate flows: 2a. The user already has every ingredient: the system reports that nothing needs to be bought.
Postconditions: The user has a grocery list of only what is missing, with an estimated cost.

UC-07: Scale Servings and Log Leftovers
Requirements: REQ-F-02, REQ-F-12, REQ-F-13.
Actor: Scrap user.
Preconditions: The user is viewing a recipe.
Main flow:

1. The user changes the number of servings.
2. The system rescales every ingredient quantity.
3. After cooking, the user logs any extra portions as leftovers.
4. The system adds the leftovers to the pantry and suggests recipes that reuse them.

Alternate flows: 3a. There are no leftovers: the user skips logging.
Postconditions: Leftovers are tracked in the pantry instead of being forgotten.

UC-08: Use Up Expiring Ingredients
Requirements: REQ-F-03, REQ-F-14, REQ-F-20, REQ-NF-02, REQ-T-07, REQ-T-10.
Actor: Scrap user.
Preconditions: At least one pantry item has an expiration date within the reminder window.
Main flow:

1. The system notifies the user that an item is about to expire.
2. The user opens the notification.
3. The system shows recipes that use the expiring item, ranked by pantry match.
4. The user selects a recipe, continuing to UC-04.

Alternate flows: 3a. No recipe uses the item: the system suggests searching the community feed or contributed recipes.
Postconditions: The user has a plan to use the ingredient before it spoils.

UC-09: Contribute a Recipe
Requirements: REQ-F-01, REQ-F-07, REQ-F-15, REQ-NF-05, REQ-C-01, REQ-C-04, REQ-T-10.
Actor: Scrap user.
Preconditions: The user is logged in.
Main flow:

1. The user selects *Add recipe*.
2. The user enters the title, ingredients, written steps, servings, time, and difficulty.
3. The user optionally attaches a photo or video.
4. The system validates that written steps are present, then publishes the recipe.

Alternate flows: 3a. The file exceeds the upload size limit: the system asks the user to shorten or compress it. 4a. Required fields are missing: the system highlights them and does not publish.
Postconditions: The recipe is searchable by other users.

UC-10: Share a Dish and Browse the Community Feed
Requirements: REQ-F-01, REQ-F-16, REQ-NF-04, REQ-C-01, REQ-F-17.
Actor: Scrap user.
Preconditions: The user is logged in.
Main flow:

1. The user selects *Share a dish* and uploads a photo.

2. The user optionally links the post to a recipe and chooses whether it is public.
3. The system publishes the post to the community feed.
4. The user browses the feed and opens linked recipes (UC-04).

Alternate flows: 2a. The user makes the post private: it is visible only on the user's own profile.
Postconditions: The user's dish is shared, and the user has seen what others are cooking.

UC-11: Add Pantry Items by Photo
Requirements: REQ-F-02, REQ-F-19, REQ-NF-01, REQ-NF-03.
Actor: Scrap user.
Preconditions: The user is logged in and viewing the pantry screen.
Main flow:

1. The user selects *Add by photo* instead of typing ingredients in manually.
2. The user takes a photo of their ingredients or uploads an existing photo.
3. The system identifies candidate ingredients from the photo and lists them for confirmation.
4. The user reviews the list, edits or removes any misidentified items, and confirms.
5. The system adds the confirmed items to the pantry, continuing UC-02.

Alternate flows: 3a. The system cannot confidently identify any items: it prompts the user to enter them manually instead. 4a. The user removes an incorrectly identified item before confirming.
Postconditions: The pantry is updated without the user typing each ingredient by hand.

UC-12: Get Notified Before an Ingredient Spoils
Requirements: REQ-F-14, REQ-F-20, REQ-NF-02, REQ-T-07, REQ-T-10.
Actor: Scrap user.
Preconditions: The user has a fresh, unpackaged pantry item (such as a vegetable) for which no expiration date was entered manually.
Main flow:

1. When the item is added, the system estimates its shelf life from its ingredient type.
2. As the estimated shelf life nears its end, the system sends a push notification.
3. The user opens the notification.
4. The system shows recipes that use the expiring item, ranked by pantry match, continuing to UC-04.

Alternate flows: 1a. The ingredient type is not recognized: the system asks the user to confirm or manually set an expiration date instead (UC-02).
Postconditions: The user is alerted and offered a recipe before the ingredient spoils, without having to track expiration dates by hand.

8.1 Activity Diagrams for Key Workflows

The two diagrams below work through the decision points behind the use cases that most depend on a branching outcome rather than a single straight-line flow: the core ingredient-match search, and the newer photo-based pantry entry added from Milestone 1 presentation feedback (see `REQ-F-19`, and Key Constraints' Design Iteration section). Notation follows OMG UML 2.5.1 activity diagrams [10]: a filled circle is the initial node, a ringed filled circle is the activity final node, rounded rectangles are actions, and diamonds are decision points.

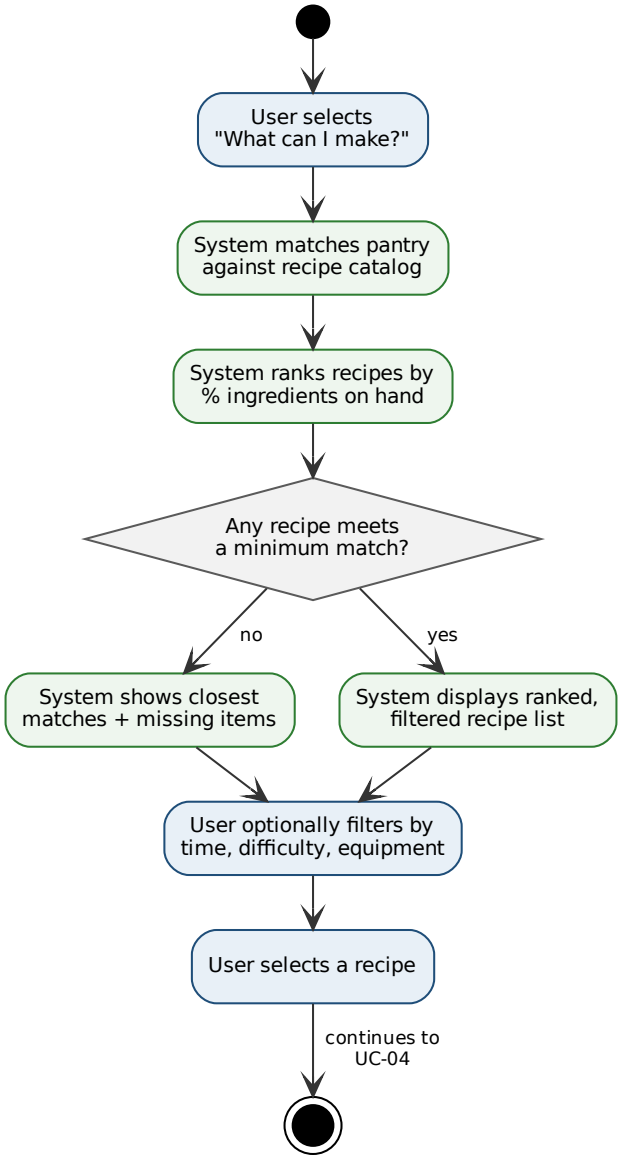


Figure 8.1: Activity diagram for UC-03 (Find Recipes from On-Hand Ingredients). Blue actions are performed by the user; green actions are performed by the system. When no recipe clears the minimum match threshold, the system falls back to showing the closest matches and what they are missing, rather than an empty result.

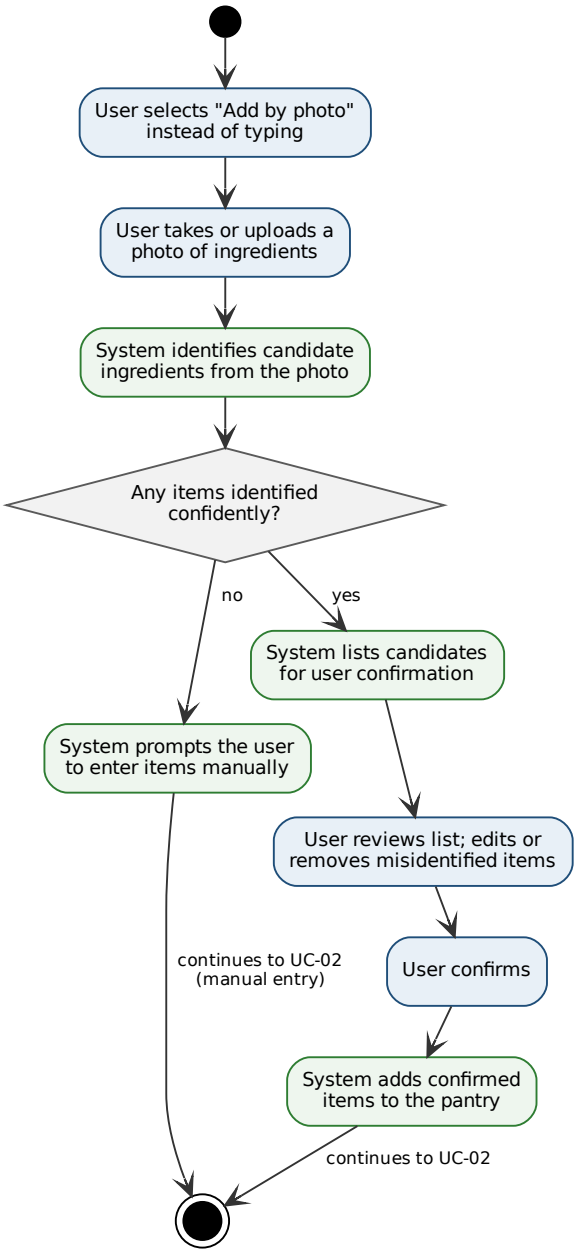


Figure 8.2: Activity diagram for UC-11 (Add Pantry Items by Photo). If the system cannot confidently identify any ingredient in the photo, the user is routed to manual entry (UC-02) instead of being left with an empty or stuck screen.

Chapter 9

Traceability Matrix

The tables below give the complete, reciprocal mapping between User Stories, Requirements, and Use Cases. Every identifier is a link to its definition. Every requirement maps to at least one use case, and every use case maps to at least one requirement, with one exception: REQ-T-13 (the companion website) shares no use cases with the app, since it is a separate product.

9.1 User Stories to Requirements

Table 9.1: User Stories to Requirements

Story	Source	Requirements
US-01	Interviewee 1	REQ-F-03, REQ-F-04, REQ-F-09, REQ-F-11, REQ-F-12
US-02	Interviewee 1	REQ-F-01, REQ-F-15, REQ-F-16
US-03	Interviewee 2	REQ-F-04, REQ-F-05, REQ-F-06, REQ-F-07, REQ-NF-01
US-04	Interviewee 2	REQ-F-02, REQ-F-12, REQ-F-13, REQ-F-16
US-05	Interviewee 3	REQ-F-02, REQ-F-03, REQ-F-06, REQ-F-08, REQ-F-10, REQ-F-14
US-06	Interviewee 3	REQ-F-05, REQ-F-07, REQ-NF-01, REQ-NF-03
US-07	Interviewee 4	REQ-F-02, REQ-F-09, REQ-F-10, REQ-F-12, REQ-F-13
US-08	Interviewee 4	REQ-F-03, REQ-F-05, REQ-F-07, REQ-F-14
US-09	Derived (developer)	REQ-T-02, REQ-T-04
US-10	Derived (developer)	REQ-T-06
US-11	Derived (developer)	REQ-T-03
US-12	Derived (business)	REQ-T-01, REQ-T-05
US-13	Derived (business)	REQ-T-07, REQ-F-14
US-14	Derived (community)	REQ-F-17

9.2 Functional Requirements to Stories and Use Cases

Table 9.2: Functional Requirements to Stories and Use Cases

Req.	Name	User stories	Use cases
REQ-F-01	User accounts	US-02	UC-01, UC-09, UC-10
REQ-F-02	Pantry inventory	US-04, US-05, US-07	UC-02, UC-05, UC-07
REQ-F-03	Ingredient-match search	US-01, US-05, US-08	UC-03, UC-08
REQ-F-04	Recipe filters	US-01, US-03	UC-03
REQ-F-05	Complete recipe view	US-03, US-06, US-08	UC-04
REQ-F-06	Recipe facts	US-03, US-05	UC-04
REQ-F-07	Recipe video	US-03, US-06, US-08	UC-04, UC-09
REQ-F-08	Ingredient substitutions	US-05	UC-05
REQ-F-09	Grocery list	US-01, US-07	UC-06
REQ-F-10	Estimated cost	US-05, US-07	UC-06
REQ-F-11	Equipment profile	US-01	UC-01, UC-03
REQ-F-12	Serving scaling	US-01, US-04, US-07	UC-07
REQ-F-13	Leftover tracking	US-04, US-07	UC-07
REQ-F-14	Expiration alerts	US-05, US-08	UC-02, UC-08
REQ-F-15	Recipe contribution	US-02	UC-09
REQ-F-16	Community feed	US-02, US-04	UC-10
REQ-F-17	Content moderation	US-14	UC-10
REQ-F-18	Barcode pantry entry	Derived	UC-02

Continued on next page

Req.	Name	User stories	Use cases
REQ-F-19	Photo-based pantry entry	Derived (presentation feedback)	UC-02, UC-11
REQ-F-20	Shelf-life estimation and reminders	Derived (presentation feedback)	UC-02, UC-08, UC-12

9.3 Non-Functional Requirements to Stories and Use Cases

Table 9.3: Non-Functional Requirements to Stories and Use Cases

Req.	Name	User stories	Use cases
REQ-NF-01	Usability	US-03, US-06	UC-03, UC-04
REQ-NF-02	Performance	Derived	UC-03, UC-08
REQ-NF-03	Mobile responsiveness	US-06	UC-02, UC-04, UC-06
REQ-NF-04	Security and privacy	Derived	UC-01, UC-10
REQ-NF-05	Accessibility	Derived	UC-04, UC-09

9.4 Constraints to Stories and Use Cases

Table 9.4: Constraints to Stories and Use Cases

Req.	Name	User stories	Use cases
REQ-C-01	Budget	Key Constraints	UC-03, UC-09, UC-10
REQ-C-02	Schedule	Key Constraints	All (UC-01–UC-10)
REQ-C-03	Team size	Key Constraints	All (UC-01–UC-10)
REQ-C-04	Data sources	Key Constraints	UC-04, UC-09

Continued on next page

Req.	Name	User stories	Use cases
REQ-C-05	Price data	Key Constraints	UC-06
REQ-C-06	Bug tracking	Derived	All (UC-01–UC-10)

9.5 Technical Requirements to Stories and Use Cases

Table 9.5: Technical Requirements to Stories and Use Cases

Req.	Name	User stories	Use cases
REQ-T-01	Native platform	US-12	All (UC-01–UC-10)
REQ-T-02	MVVM architecture	US-09	UC-03, UC-06
REQ-T-03	Offline-first persistence	US-11	UC-02, UC-03
REQ-T-04	API and networking	US-09	All (UC-01–UC-10)
REQ-T-05	Platform guidelines	US-12	All (UC-01–UC-10)
REQ-T-06	Automated testing	US-10	UC-01, UC-03, UC-06
REQ-T-07	Push notifications	US-13	UC-08
REQ-T-08	Backend platform	Derived	All (UC-01–UC-10)
REQ-T-09	Database schema	Derived	UC-02, UC-03, UC-06
REQ-T-10	Recipe data source	Derived	UC-03, UC-04, UC-08, UC-09
REQ-T-11	Third-party API resilience	Derived	UC-03, UC-04
REQ-T-12	Beta distribution	Derived	All (UC-01–UC-10)
REQ-T-13	Companion website	Derived	N/A (separate product)

9.6 Use Cases to Requirements

Table 9.6: Use Cases to Requirements

Use case	Name	Requirements
UC-01	Create Account and Kitchen Profile	REQ-F-01, REQ-F-11, REQ-NF-04, REQ-T-06
UC-02	Manage Pantry	REQ-F-02, REQ-F-14, REQ-F-18, REQ-NF-03, REQ-T-03, REQ-T-09
UC-03	Find Recipes from On-Hand Ingredients	REQ-F-03, REQ-F-04, REQ-F-11, REQ-NF-01, REQ-NF-02, REQ-C-01, REQ-T-02, REQ-T-03, REQ-T-06, REQ-T-09, REQ-T-10, REQ-T-11
UC-04	View Recipe Details	REQ-F-05, REQ-F-06, REQ-F-07, REQ-NF-01, REQ-NF-03, REQ-NF-05, REQ-C-04, REQ-T-10, REQ-T-11
UC-05	Substitute a Missing Ingredient	REQ-F-02, REQ-F-08
UC-06	Generate Grocery List	REQ-F-09, REQ-F-10, REQ-NF-03, REQ-C-05, REQ-T-02, REQ-T-06, REQ-T-09
UC-07	Scale Servings and Log Leftovers	REQ-F-02, REQ-F-12, REQ-F-13
UC-08	Use Up Expiring Ingredients	REQ-F-03, REQ-F-14, REQ-NF-02, REQ-T-07, REQ-T-10
UC-09	Contribute a Recipe	REQ-F-01, REQ-F-07, REQ-F-15, REQ-NF-05, REQ-C-01, REQ-C-04, REQ-T-10
UC-10	Share a Dish and Browse the Community Feed	REQ-F-01, REQ-F-16, REQ-NF-04, REQ-C-01, REQ-F-17
UC-11	Add Pantry Items by Photo	REQ-F-02, REQ-F-19, REQ-NF-01, REQ-NF-03
UC-12	Get Notified Before an Ingredient Spoils	REQ-F-14, REQ-F-20, REQ-NF-02, REQ-T-07, REQ-T-10

Chapter 10

Verification, Validation, and Testing

This chapter defines how the team will confirm that Scrap is built correctly and that it solves the problem it set out to solve. Verification asks whether the system conforms to its own specification — *did we build the system right* — and is checked per requirement through inspection, demonstration, test, or analysis [11]. Validation asks whether the completed system meets real stakeholder needs in its intended context — *did we build the right system* — and is checked through realistic usage scenarios with actual users [11]. Because Scrap has not yet been built, every status below is **PENDING**; the team will update this chapter as the Development Plan chapter's Testing Plan is carried out, and validation in particular will not be meaningful until beta users are recruited in January 2027 (see that chapter's Timeline and Milestones).

10.1 Verification Register

The register below lists a representative set of Scrap's requirements, including the two bonus requirements added from Milestone 1 presentation feedback (`REQ-F-19`, `REQ-F-20`) and the three requirements that come directly from an external code or standard (`VER-013` through `VER-015`; see the Codes and Standards chapter), the verification method that applies to each, and the evidence the team will collect. It is not exhaustive of all requirements in Requirements; it will grow as implementation proceeds.

Table 10.1: Verification Register

ID	Requirement	Method	Required Evidence	Status
`VER-001`	`REQ-F-01`	Test	XCUITest run covering account creation, login, and logout	**PENDING**
`VER-002`	`REQ-F-02`	Test	XCTest unit tests for adding, editing, and removing pantry items	**PENDING**

Continued on next page

ID	Requirement	Method	Required Evidence	Status
VER-003	REQ-F-03	Test	XCTest unit tests comparing ranked output to expected results for fixture pantries and recipes (TC-001)	PENDING
VER-004	REQ-F-09	Test	XCTest unit test confirming a generated grocery list contains only missing ingredients (TC-002)	PENDING
VER-005	REQ-F-19	Demonstration	Manual walkthrough on a device: photograph a set of known ingredients and confirm the proposed pantry items (TC-004)	PENDING
VER-006	REQ-F-20	Test	Unit tests on the shelf-life estimation logic plus a demonstrated notification for an ingredient nearing its estimated shelf life (TC-005)	PENDING
VER-007	REQ-NF-01	Inspection	UI walkthrough confirming a recipe's full ingredient list and instructions are reachable within one tap from the results list	PENDING
VER-008	REQ-NF-02	Test	Automated timing test returning ranked results for a 100-item pantry fixture in under 2 seconds (TC-003)	PENDING
VER-009	REQ-NF-04	Inspection	Code review confirming Supabase authentication stores only salted password hashes and posts respect the user's public/private choice	PENDING
VER-010	REQ-C-01	Analysis	Review of projected Spoonacular call volume and Supabase storage against free-tier limits	PENDING

Continued on next page

ID	Requirement	Method	Required Evidence	Status
VER-011	REQ-T-06	Inspection	Review of the XCTest/XCUITest coverage report for ingredient-matching, grocery-list, account, and search flows	PENDING
VER-012	REQ-T-11	Test	Simulated Spoonacular error and rate-limit responses confirmed not to crash the app or show a blank screen (TC-006)	PENDING
VER-013	REQ-NF-03	Inspection	UI walkthrough on the narrowest supported simulator width (375 pt) confirming every screen is fully usable with no horizontal scrolling or clipped controls	PENDING
VER-014	REQ-NF-05	Inspection	VoiceOver walkthrough of every screen checked against WCAG 2.1 Level AA [7] (contrast, Dynamic Type, focus order, labeled controls), plus confirmation that every recipe video is accompanied by complete written steps	PENDING
VER-015	REQ-T-05	Inspection	Design review against Apple's Human Interface Guidelines [8] (navigation, typography, Dynamic Type) and a pre-submission check against the App Store Review Guidelines [9] before any TestFlight or App Store submission	PENDING

10.2 Validation Register

Validation exercises Scrap with real people once a working prototype exists, starting with the beta users recruited in January 2027. Each scenario below is tied to the user story that motivated it.

Table 10.2: Validation Register

ID	Related Story	Scenario	Success Criterion	Status
VAL-001	US-01	A beta user with a stocked pantry opens Scrap and searches for what they can make.	The user finds and cooks a recipe made mostly from ingredients already on hand, without leaving the app.	PENDING
VAL-002	US-05	A budget-conscious beta user relies on Scrap instead of shopping for a recipe from scratch for a week.	The user reports spending less and using more of what they already owned.	PENDING
VAL-003	US-08	A beta user adds pantry items by photo instead of typing them in.	The user completes pantry entry faster than manual typing and confirms the identified items were correct.	PENDING
VAL-004	US-13	A beta user receives a shelf-life reminder for a vegetable nearing spoilage.	The user opens the notification and cooks a suggested recipe before the ingredient spoils.	PENDING
VAL-005	US-02	A beta user shares a dish they cooked to the community feed.	The user reports that sharing feels social rather than like a chore.	PENDING
VAL-006	US-14	A beta user reports an inappropriate community post.	A moderator reviews and removes the content within an agreed response time.	PENDING
VAL-007	US-03	A beta user who normally finds recipes on social media opens a recipe in Scrap that includes a video.	The user follows the written steps directly in the app, without needing to replay or scrub the video to find a step.	PENDING
VAL-008	US-04	A beta user logs leftover portions from a cooked recipe.	Scrap suggests a recipe that uses the logged leftovers before they would otherwise be forgotten and go to waste.	PENDING

Continued on next page

ID	Related Story	Scenario	Success Criterion	Status
VAL-009	US-06	A beta user browsing recipes on their phone opens a recipe from the results list.	The user reaches the complete ingredient list and instructions within one tap, without leaving the app or hunting for hidden content.	PENDING
VAL-010	US-07	A budget-conscious beta user generates a grocery list for a recipe they are missing a few ingredients for.	The list contains only the ingredients the user actually lacks, scaled to their chosen servings, with an estimated total cost.	PENDING

10.3 Example Test Cases

The test cases below give concrete, repeatable procedures for a sample of the verification rows above, following a standard test-case structure of mapped requirement, procedure, and expected result [12]. Evidence from running these (screenshots, console output, or CI logs) will be recorded once implementation begins.

Table 10.3: Example Test Cases

ID	Mapped Requirement	Procedure	Expected Result
TC-001	REQ-F-03	Load a fixture pantry and recipe catalog, call the ranking function, and compare the output order to the expected order.	Recipes are ranked by descending percentage of matched ingredients.
TC-002	REQ-F-09	Select a recipe with two missing ingredients and generate a grocery list.	The list contains exactly the two missing ingredients, scaled to the chosen number of servings.
TC-003	REQ-NF-02	Populate the pantry with 100 items and request ranked recipes, measuring response time.	Results are returned within 2 seconds.

Continued on next page

ID	Mapped Requirement	Procedure	Expected Result
TC-004	REQ-F-19	Upload a test photo containing three known ingredients.	The system proposes three matching candidate items for the user to confirm.
TC-005	REQ-F-20	Add a fresh vegetable with no manual expiration date, then advance the estimated shelf life to its end.	The app sends a push notification suggesting a recipe that uses that ingredient.
TC-006	REQ-T-11	Simulate a Spoonacular rate-limit response during a recipe search.	The app shows a graceful error state rather than crashing or showing a blank screen.

Chapter 11

Developer Requirements and Coding Standards

This chapter fixes how Scrap's code is written before the team begins building in earnest, so that all three members produce consistent, compatible code rather than settling conventions ad hoc. It covers the app's MVVM architecture pattern, the namespace (module and folder) structure, class and type naming conventions, and the UML diagrams that lay out the app's structure. Each rule is stated as a developer requirement (REQ-D) in the same "shall" form used in the Requirements chapter. All REQ-D requirements are derived by the team from the developer stakeholder stories US-09 (isolated business logic) and US-10 (confidence to change core logic), from the technical requirements REQ-T-01 through REQ-T-11, and from the ScrapCore/ScrapApp split described in the Development Plan's Risks and Assumptions section. Screen and feature names are taken from the team's annotated UI designs.

How to use this chapter

Every pull request into `main` is reviewed against these rules. If a rule turns out to be wrong in practice, the team changes the rule here first (with a Document History entry) and then changes the code — the code is never allowed to drift silently away from this chapter.

11.1 Architecture Pattern: MVVM Across Two Modules

Scrap uses Model–View–ViewModel (MVVM). The team develops and compiles the app in Xcode on a Mac borrowed from the professor, but that is a single machine shared by three people. The pattern is therefore split across two Swift modules: **ScrapCore**, a platform-independent Swift package that every team member can build and test on their own Windows laptop without waiting for the Mac, and **ScrapApp**, the SwiftUI app target that is built and run on the borrowed Mac. The layers and where they live are summarized below; Figure 11.1 shows the resulting package structure.

Layer	Module	Responsibility
Model	ScrapCore	Plain value types (`struct`/`enum`) describing Scrap's data: recipes, pantry items, grocery lists, and so on. No behavior beyond simple computed properties.
Domain services	ScrapCore	All business logic: ingredient matching and ranking, filtering, substitutions, serving scaling, grocery-list building, cost estimation, guest/account access rules. Pure, synchronous functions where possible.
Repository protocols	ScrapCore	Interfaces for loading and saving data (`RecipeRepository`, `KitchenRepository`, ...). No knowledge of Supabase or SwiftData.
Networking	ScrapCore	The Spoonacular client and its response types (DTOs), built on Foundation's `URLSession`.
Data adapters	ScrapApp	Concrete repositories that talk to Supabase, SwiftData, and UserNotifications.
ViewModel	ScrapApp	One per screen. Holds screen state, calls domain services and repositories, exposes intent methods to the view. Contains no business rules of its own.
View	ScrapApp	SwiftUI layout only. Renders view-model state and forwards user actions as intent calls.

REQ-D-01: Business logic lives in ScrapCore. Ingredient matching and ranking, recipe filtering, substitution lookup, serving scaling, grocery-list generation, cost estimation, and guest/account access rules shall be implemented in ScrapCore as domain services. View models shall call these services rather than re-implementing their logic.
Source: US-09, REQ-T-02; shared-Mac constraint (Development Plan). **Rationale:** keeps the logic testable without the shared Mac and reachable through a single documented API.

REQ-D-02: ScrapCore is platform-independent. ScrapCore shall import only the Swift standard library and Foundation (including `FoundationNetworking` where required). It shall not import SwiftUI, UIKit, SwiftData, Combine, or any other Apple-only framework, and `swift build` and `swift test` shall succeed for ScrapCore on Windows. If a third-party SDK (for example, the Supabase Swift SDK) does not build on Windows, ScrapCore shall contain only the protocol, and the SDK-based implementation shall live in ScrapApp.
Source: Shared-Mac constraint (Development Plan), REQ-T-06.

REQ-D-03: One-way dependencies. Dependencies shall point in one direction only: View → ViewModel → domain services and repository protocols → Models. ScrapCore shall never depend on ScrapApp; a view shall never call a repository, a network client, or Supabase directly; and a model type shall never reference a view model or view.
Source: US-09, REQ-T-02.

REQ-D-04: View model shape. Each screen shall have exactly one view model, declared as a `@MainActor @Observable final class`. Its dependencies (repositories and services) shall be passed in through its initializer as protocol types, never created inside it. View models shall not import SwiftUI; they expose state as stored properties and user actions as methods named for the intent (for example `addToGroceryList(_:)`, not `buttonTapped()`).
Source: US-09, US-10, REQ-T-02.

REQ-D-05: Explicit screen state. Any screen that loads data shall represent its state with the shared generic enum `ViewState<Value>` with the cases `idle`, `loading`, `loaded(Value)`, `empty`, and `failed(ScrapError)`, and every view shall render all five cases. No screen may show a blank area for the `empty` or `failed` cases.
Source: REQ-T-11. **Rationale:** makes "no crash, no blank screen" on a bad Spoonacular response a structural guarantee rather than something each developer must remember.

REQ-D-06: Dependency wiring in one place. All concrete repositories and clients shall be created in a single `AppDependencies` type in ScrapApp at launch and passed down to view models. SwiftUI previews and tests shall use in-memory implementations (see REQ-D-16) instead of live services.
Source: US-10, REQ-T-06.

REQ-D-07: Offline-first repositories. Repositories that back pantry and recipe data shall read from the local cache first and refresh from Supabase or Spoonacular in the background. The cache-then-network logic shall live inside the repository implementation, so view models never contain code paths for "online" versus "offline".
Source: US-11, REQ-T-03.

REQ-D-08: Guest versus account access in one place. The difference between guest users and signed-in users (for example, guests cannot save recipes, comment, or keep a grocery list past the current day, and are prompted to create an account instead) shall be decided only by an `AccessPolicy` service in ScrapCore that takes the current `SessionState`. Views and view models shall ask `AccessPolicy` rather than checking login status themselves.
Source: Team UI review (guest/account behavior on every screen), REQ-F-01.

11.2 Namespaces

Swift has no `namespace` keyword. Each *module* is a namespace (a type can always be written in full as `ScrapCore.Recipe`), and within a module the team uses folders and, for grouped constants, caseless enums. The requirements below define Scrap's namespaces and how code is placed in them.

REQ-D-09: Two top-level namespaces. Scrap's code shall be organized into exactly two first-party modules, `ScrapCore` (Swift package) and `ScrapApp` (Xcode app target), plus their test targets `ScrapCoreTests` and `ScrapAppUITests`. A new module shall be added

only by updating this chapter first.
Source: Shared-Mac constraint (Development Plan), REQ-T-06.

REQ-D-10: No type shares a module's name. No type shall be named `ScrapCore` or `ScrapApp`, because a type with the same name as its module breaks fully qualified names in Swift. The SwiftUI entry point shall therefore be `@main struct ScrapMain: App`. Types shall also not carry a redundant `Scrap` prefix (`Recipe`, not `ScrapRecipe`); the module already provides that context. The only exception is the shared error type `ScrapError`.
Source: Derived by the team (Swift module behavior).

REQ-D-11: Folder structure is the namespace structure. Source files shall be placed in the folder structure below. ScrapCore is organized by *domain area*; ScrapApp is organized by *feature (screen)*, with one folder per screen from the team's UI designs. A file shall not be placed in a folder whose name does not describe it, and folders named `Utils`, `Helpers`, `Misc`, or `Managers` shall not be created.
Source: US-09; team UI designs.

```
ScrapCore/                          (Swift package; builds on any machine)
  Package.swift
  Sources/ScrapCore/
    Models/          Recipe.swift, RecipeIngredient.swift, Ingredient.swift,
                     PantryItem.swift, Kitchen.swift, GroceryList.swift,
                     Quantity.swift, Equipment.swift, DietaryRestriction.swift,
                     MealType.swift, Cuisine.swift, RecipeSubmission.swift, ...
    Matching/        RecipeMatcher.swift, RecipeFilter.swift,
                     RecipeFilterOptions.swift, SubstitutionFinder.swift
    GroceryList/     GroceryListBuilder.swift, CostEstimator.swift
    Scaling/         ServingScaler.swift
    Access/          AccessPolicy.swift, SessionState.swift
    Repositories/    RecipeRepository.swift, KitchenRepository.swift,
                     GroceryListRepository.swift, CommunityRepository.swift,
                     AuthService.swift               (protocols only)
    Networking/      SpoonacularRecipeClient.swift, SpoonacularEndpoint.swift,
                     SpoonacularRecipeDTO.swift
    Common/          ViewState.swift, ScrapError.swift
  Tests/ScrapCoreTests/
    RecipeMatcherTests.swift, GroceryListBuilderTests.swift, ...

ScrapApp/                           (Xcode app target; built on the borrowed Mac)
  App/               ScrapMain.swift, AppDependencies.swift
  Features/
    Auth/            SignInView, SignUpView, CreateAccountPromptView (+ VMs)
    Home/            HomeView, HomeViewModel   (search bar + top recipes)
    RecipeDetail/    RecipeDetailView, RecipeDetailViewModel,
                     SubstitutionSheet
    Filters/         RecipeFiltersSheet, RecipeFiltersViewModel
    GroceryList/     GroceryListView, GroceryListViewModel
    Kitchen/         KitchenView, KitchenViewModel, PantrySection,
                     FridgeSection, EquipmentPicker, RestrictionToggles
    ReceiptUpload/   ReceiptUploadView, ReceiptUploadViewModel
```

```
    Community/      FeedView, FeedViewModel, FriendsView, FriendsViewModel
    Profile/        ProfileView, ProfileViewModel, NotificationSettingsView
  Data/             SupabaseKitchenRepository, CachedRecipeRepository,
                    SwiftDataRecipeCache, LocalNotificationScheduler, ...
  DesignSystem/     ScrapColor, ScrapFont, NotebookPaperStyle
  Resources/        Assets.xcassets, Localizable.xcstrings
ScrapAppUITests/
```

REQ-D-12: Caseless enums for grouped constants. Related constants and static helpers that do not belong to a model shall be grouped in a caseless `enum` that acts as a namespace (see the example below). Global free-standing constants and global functions shall not be used. API keys and Supabase URLs shall not be written in source code; they shall be read from a configuration file excluded from Git.
Source: Derived by the team; REQ-T-04.

```
enum SpoonacularEndpoint {
    static let searchByIngredients = "/recipes/findByIngredients"
    static let recipeInformation = "/recipes/{id}/information"
}
```

REQ-D-13: Nested types for type-specific helpers. A type used by only one other type shall be nested inside it (for example `RecipeFilterOptions.SortOrder`, `GroceryListItem.Source`) rather than declared at the top level of the module.
Source: Derived by the team.

REQ-D-14: Access control defines the public API. Everything in ScrapCore shall default to `internal`. Only the types and members ScrapApp actually needs shall be marked `public`, and every `public` declaration shall have a `///` documentation comment. This public surface is the "documented API" that US-09 asks for. Inside ScrapApp, view-model properties that views must not change shall be `private(set)`.
Source: US-09, REQ-T-02.

11.3 Class and Type Naming Conventions

Scrap follows Apple's Swift API Design Guidelines. The requirements below add the project-specific rules.

REQ-D-15: Base naming rules. Types and protocols shall use `UpperCamelCase`; properties, functions, enum cases, and local variables shall use `lowerCamelCase`. Acronyms shall be cased uniformly (`recipeID`, `imageURL`, `SpoonacularDTO`; never `recipeId`). Booleans shall read as assertions (`isChecked`, `hasAccount`, `canSaveRecipes`). Names shall not use abbreviations other than `ID`, `URL`, and `DTO`.
Source: Derived by the team (Swift API Design Guidelines).

REQ-D-16: Role suffixes and prefixes. A type's name shall state its role using the suffixes and prefixes in the table below, and each source file shall contain one primary type and be named after it (`RecipeMatcher.swift`). Extensions shall be placed in files named `Type+Purpose.swift` (for example `Recipe+Sample.swift`).
Source: US-09, US-10.

Kind of type	Pattern	Examples
Model	Plain noun, singular	`Recipe`, `PantryItem`, `GroceryList`
SwiftUI screen	`<Screen>View`	`HomeView`, `KitchenView`
Sheet or reusable sub-view	`<Thing>Sheet`, `<Thing>Row`, `<Thing>Card`, `<Thing>Section`	`SubstitutionSheet`, `RecipeCard`, `PantrySection`
View model	`<Screen>ViewModel` (matches its view)	`HomeViewModel`, `GroceryListViewModel`
Domain service	Agent noun ending in *-er*, or a clear noun	`RecipeMatcher`, `ServingScaler`, `GroceryListBuilder`, `AccessPolicy`
Repository protocol	`<Entity>Repository`	`RecipeRepository`, `KitchenRepository`
Repository implementation	`<Technology><Entity>Repository`	`SupabaseKitchenRepository`, `CachedRecipeRepository`
Other service protocol	Noun for the capability	`AuthService`, `NotificationScheduler`, `ReceiptParser`
External API client	`<Service>Client`	`SpoonacularRecipeClient`
API response type	`<Service><Thing>DTO`	`SpoonacularRecipeDTO`
Error	`<Area>Error` (enum)	`ScrapError`, `NetworkError`
Test doubles	`InMemory<Protocol>`, `Mock<Protocol>`, `Stub<Protocol>`	`InMemoryKitchenRepository`, `StubRecipeRepository`
Unit test class	`<TypeUnderTest>Tests`	`RecipeMatcherTests`

REQ-D-17: Shared domain vocabulary. Code, database tables, and UI copy shall use the terms in the table below with exactly these meanings. A new domain term shall be added to this table (and to the Glossary) before it is used in code.
Source: Team UI review; REQ-T-09.

Term (type)	Meaning
`Kitchen`	Everything a user has: their pantry and fridge items, equipment, and dietary restrictions. Replaces "My Pantry" as the umbrella term (the UI's "My Kitchen").
`PantryItem`	One ingredient the user has on hand. Its `location` is `.pantry` (non-perishables, frozen items, spices; no expiration date needed) or `.fridge` (perishables, usually added from a receipt, with an estimated expiration date).

`Ingredient`	A canonical ingredient identity (Spoonacular ID and name). Pantry items and recipe ingredients both point to one, which is how matching works.
`RecipeIngredient`	An ingredient as it appears in a recipe, with a `Quantity`.
`Equipment`	A kitchen appliance or tool. Equipment a recipe needs but the user lacks counts as a *missing item*, just like a missing ingredient.
`RankedRecipe`	A recipe paired with its `matchPercent`, missing ingredients, missing equipment, and estimated cost of the missing items. Produced only by `RecipeMatcher`.
`matchPercent`	Share of a recipe's ingredients the user already has, as a `Double` from 0.0 to 1.0 (formatted as a percentage only in the view).
`Substitution`	A replacement for a missing ingredient, flagged `isInKitchen` when the user already has it.
`GroceryList`	A list of items to buy. A guest's list has `ownerID == nil` and is not saved past the current day.
`RecipeSubmission`	A user-contributed recipe awaiting admin approval (`pending`, `approved`, `rejected`).
`SessionState`	`.guest` or `.signedIn(User)`; the only input to `AccessPolicy`.

REQ-D-18: Database naming and mapping. Supabase tables shall be plural `snake_case` (`pantry_items`, `recipe_ingredients`, `grocery_list_items`); columns shall be `snake_case`; foreign keys shall be named `<singular_table>_id` (`recipe_id`). Swift code shall use the camelCase equivalent, converted by the decoder's snake-case key strategy or explicit `CodingKeys`. Database rows and API responses shall be converted into ScrapCore model types at the repository or client boundary, so DTO types never reach a view model.
Source: REQ-T-09, REQ-T-10.

11.4 Code Style, Errors, and Concurrency

REQ-D-19: Automatic formatting. All Swift code shall be formatted with `swift format` (bundled with the Swift 6 toolchain) using one shared `.swift-format` configuration file committed to the repository root: 4-space indentation, a 120-character line limit, and trailing commas in multi-line collections. Code shall be formatted before every commit.
Source: Derived by the team.

REQ-D-20: Value types by default. Models shall be `structs` or `enums` that conform to `Equatable`, `Hashable` where useful, `Identifiable` where they appear in lists, `Codable` where they are stored or transmitted, and `Sendable`. Classes shall be used only for view models and for repository or client implementations that hold shared state, and every class shall be `final` unless it is designed for subclassing.
Source: Derived by the team.

REQ-D-21: Async/await only. Asynchronous work shall use Swift concurrency (`async/await`, `Task`). Completion-handler callbacks and Combine pipelines shall not be

introduced. Repository and client methods that can fail shall be `async throws`.
Source: Derived by the team; REQ-T-04.

REQ-D-22: Errors are typed and never crash the app. Production code shall not use `try!`, force unwrapping (`!`), `fatalError`, or force casts (`as!`). Errors shall be mapped to a case of `ScrapError` (for example `.network`, `.rateLimited`, `.notFound`, `.unauthorized`, `.invalidData`) at the repository or client boundary, and view models shall turn them into `ViewState.failed`. A Spoonacular response with a missing field shall be decoded with an optional or default value rather than failing the whole response.
Source: REQ-T-11.

REQ-D-23: No hard-coded user-facing text or styling. User-facing strings shall live in the string catalog (`Localizable.xcstrings`), and colors and fonts shall come from `DesignSystem` (for example `ScrapColor.missingIngredient` for the red missing-ingredient highlight, `NotebookPaperStyle` for the grocery list). Fonts shall support Dynamic Type.
Source: REQ-T-05; team UI review.

11.5 Testing and Version Control

REQ-D-24: Tests accompany core logic. Every `public` domain service in ScrapCore shall have a matching `<Type>Tests` file, and a pull request that changes a domain service shall add or update its tests. Test methods shall be named `test_<method>_<condition>_<expectedResult>` (for example `test_rank_whenKitchenIsEmpty_returnsZeroMatchForAll`). Unit tests shall use in-memory or stub repositories and never call Supabase or Spoonacular.
Source: US-10, REQ-T-06.

REQ-D-25: Branches, commits, and reviews. Work shall be done on branches named `<type>/<issue-number>-<short-description>` (for example `feature/14-grocery-list-export`, `fix/22-empty-search-crash`), where `<type>` is `feature`, `fix`, `docs`, or `chore`. Commit messages shall start with an imperative summary under 72 characters ("Add serving scaler", not "added stuff"). Code shall reach `main` only through a pull request that references its GitHub Issue, passes `swift test` for ScrapCore, and has been approved by at least one other team member. A pull request that changes ScrapApp shall also be built and run in the iOS Simulator on the borrowed Mac before it is merged, so that `main` always compiles in Xcode.
Source: REQ-C-06; Workspace plan (Development Plan).

11.6 UML Diagrams of the App's Structure

The diagrams below describe the planned structure and are the reference for REQ-D-01 through REQ-D-18. Their Graphviz and LaTeX sources are kept in the repository under

`docs/uml/`.

REQ-D-26: UML diagrams stay current. The package diagram, domain class diagram, MVVM feature-slice diagram, and search sequence diagram shall be kept as source files in `docs/uml/`. A pull request that adds a module, a top-level folder, a model type, or a repository protocol shall update the affected diagram in the same pull request.
Source: US-09.

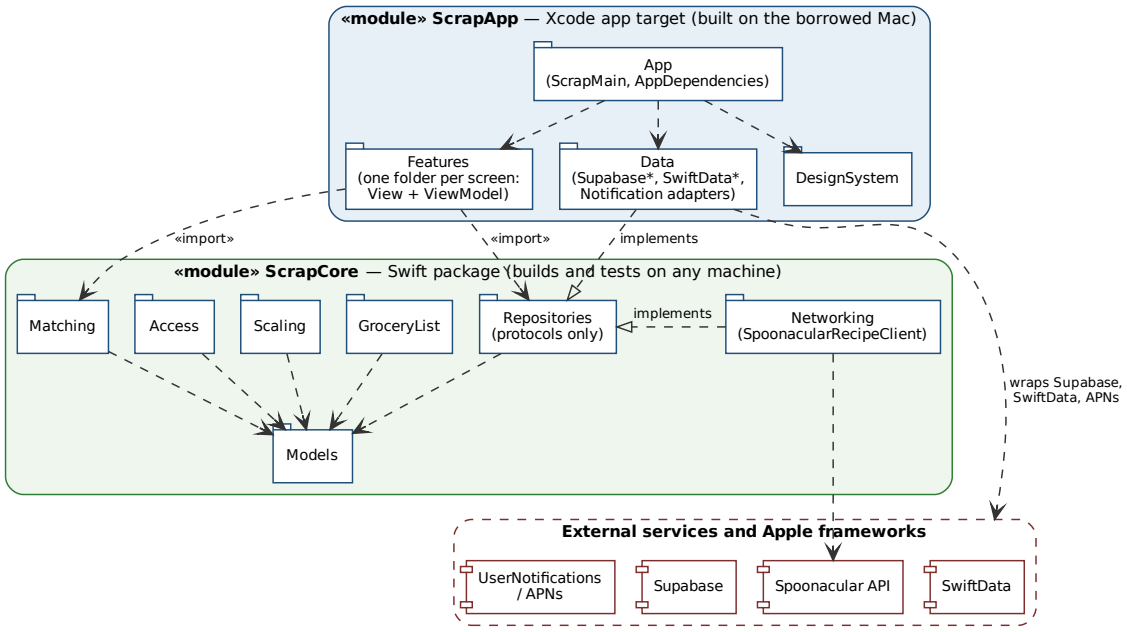


Figure 11.1: UML package diagram. ScrapApp depends on ScrapCore, never the reverse; only ScrapApp's Data adapters and ScrapCore's Networking folder touch external services.

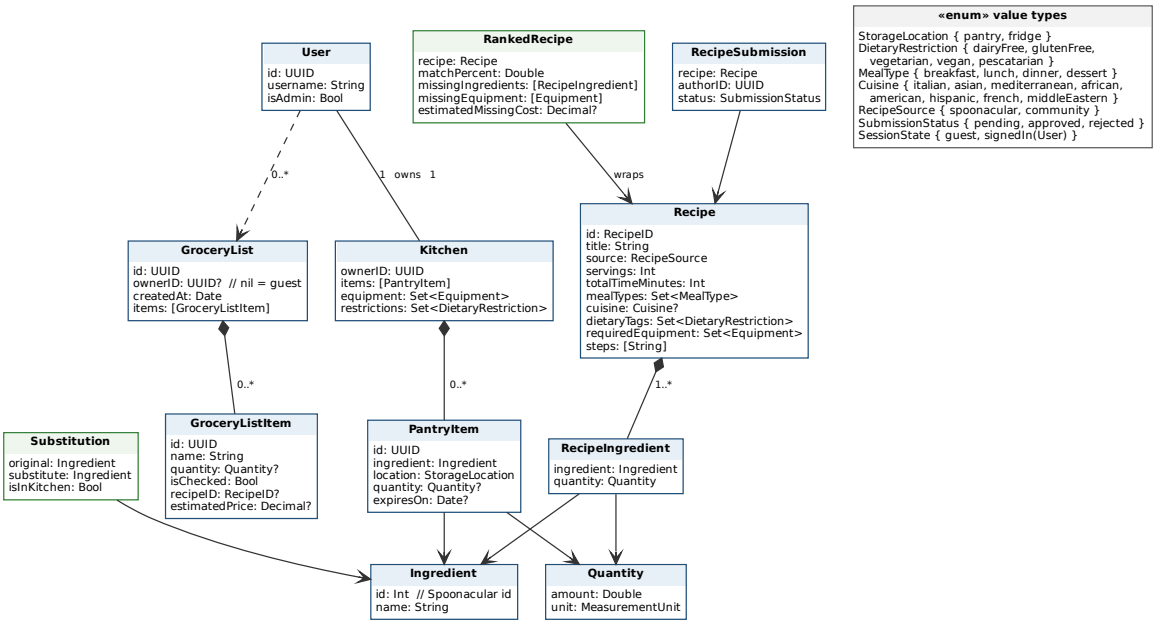


Figure 11.2: UML class diagram of the ScrapCore domain model. Green classes are computed results produced by domain services rather than stored data.

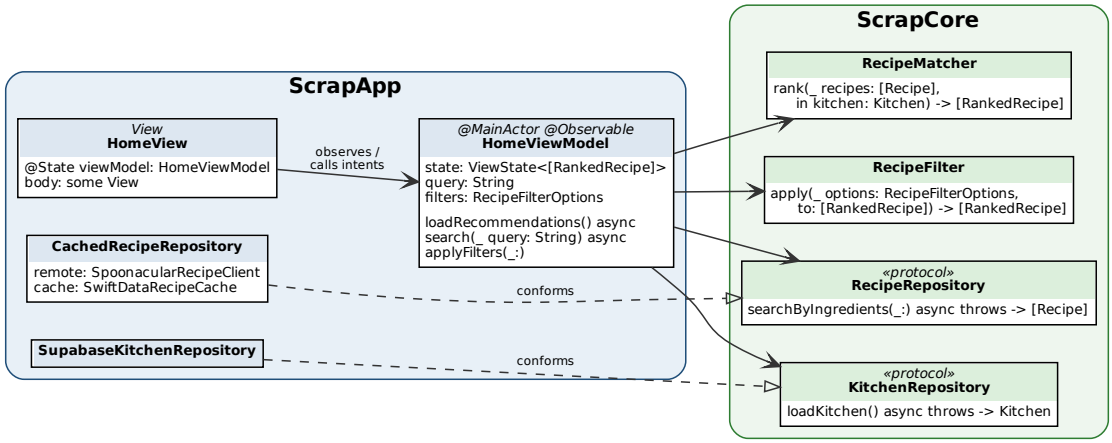


Figure 11.3: UML class diagram of one MVVM feature slice (the Home search screen). The view model depends only on ScrapCore protocols and services; concrete repositories are injected by `AppDependencies`.

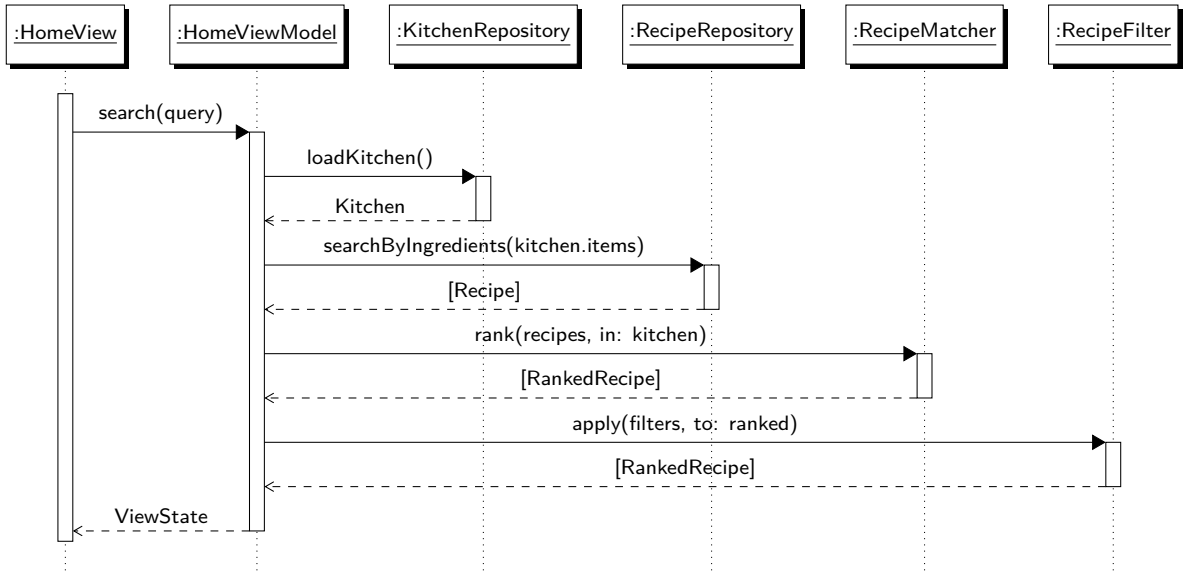


Figure 11.4: UML sequence diagram for an ingredient search (UC-03). The returned `ViewState` is `.loaded`, `.empty`, or `.failed`, so the view never shows a blank screen (REQ-D-05).

Chapter 12

Codes and Standards

This chapter consolidates every code and standard that applies to Scrap in one place, rather than leaving them as scattered citations across other chapters. Two kinds apply. **Process and documentation standards** (below) govern how this report and the team's development process are organized; they do not appear as individual `REQ` entries, but they shaped the structure of the Requirements, Use Cases, Developer Requirements, and Verification, Validation, and Testing chapters. **Product-facing standards** (below) are guidelines Scrap itself must satisfy to be usable and store-approvable; each one was written directly into Requirements as a `shall` requirement, and each has a corresponding row in the Verification Register in Verification, Validation, and Testing.

12.1 Process and Documentation Standards

These standards are not requirements Scrap must satisfy as a product; they are the process standards the team adopted to organize this report and the development work behind it.

Table 12.1: Process and Documentation Standards

Standard	Where It Applies	How It Shaped Scrap
ISO/IEC/IEEE 15288:2023, System Life Cycle Processes [13]	This report's overall structure	Organizes the report as a single life cycle — mission and drivers, requirements, design, and verification and validation — rather than disconnected sections written independently.
ISO/IEC/IEEE 12207:2026, Software Life Cycle Processes [14]	How the software-specific chapters relate	Governs how Requirements, Use Cases, and Developer Requirements build on one another rather than standing alone.

Continued on next page

Standard	Where It Applies	How It Shaped Scrap
ISO/IEC/IEEE 29148:2018, Requirements Engineering [15]	Requirements chapter	Basis for stating every requirement in *shall*/*could* form, sourced from a user story or named rationale, and independently verifiable; the structural basis for the Traceability Matrix.
IEEE 1012-2024, System, Software, and Hardware Verification and Validation [11]	Verification, Validation, and Testing chapter	Defines the verification ("did we build it right") versus validation ("did we build the right thing") split that chapter is organized around.
ISO/IEC/IEEE 29119-2:2021, Software Testing — Test Processes [12]	Example Test Cases table	Structures every test case as a mapped requirement, a procedure, and an expected result.
OMG UML 2.5.1 [10]	Every UML diagram in the report	Notation and semantics for the package, domain class, MVVM feature-slice, and search sequence diagrams in Developer Requirements, and the two activity diagrams in Use Cases.

12.2 Product-Facing Standards and Guidelines

These are guidelines Scrap itself must meet. The team identified them early as non-negotiable rather than aspirational — an iOS app that fails Apple's App Store Review Guidelines does not ship at all, and one that fails WCAG 2.1 AA is not usable by a share of its intended users — so each became a `shall` requirement in Requirements with its own planned verification, rather than a goal mentioned in passing.

Table 12.2: Product-Facing Standards and Guidelines

Standard	Applicable Scope	Design Req.	Checked By
Web Content Accessibility Guidelines (WCAG) 2.1, Level AA [7]	Every screen in the app	REQ-NF-05	VER-014

Continued on next page

Standard	Applicable Scope	Design Req.	Checked By
Apple Human Interface Guidelines [8]	Navigation, typography, and Dynamic Type support across every screen	REQ-T-05	VER-015
Apple App Store Review Guidelines [9]	Anything submitted to TestFlight or the App Store	REQ-T-05	VER-015

REQ-T-05 covers both Apple guidelines in one requirement because they were adopted together, from the same source (US-12, the business stakeholder's need for a native, approvable app) and are checked together in the same design review before any TestFlight or App Store submission (VER-015).

Part II

Appendices

Appendix A

Weekly Reports

A.1 Week Report 9 (October 28, 2026)

1. What We Did This Past Week

This week, we shifted our focus to our design review: we finished building out our Milestone 1 design review slide deck, covering our user stories and planned validation, requirements and planned verification, use cases and design diagrams, codes and standards, and constraints and design iteration; practiced delivering it together as a team; and presented it. We also started work on our Milestone 2 presentation. We did not make progress on the Mac-access decision or on real development this week.

2. What We Will Do Next Week

Next week, our goal is to finally get into tangible development: commit as a team to one of the Mac-access options we have been weighing since Week 8, set up the real Xcode project, and start writing and testing our first screens and view models against the conventions in our Developer Requirements chapter — not just the setup, but real code.

3. Action Items

- Decide as a team which Mac-access option to commit to (shared or dedicated cloud Mac vs. buying a personal Mac).
- Set up the real Xcode project and confirm ScrapCore builds inside it.
- Begin implementing and testing our first screens and view models following the Developer Requirements chapter's conventions, not just the project setup.

4. Issues and Risks

Real implementation is now a third week behind our original Week 7 plan, since this week's effort went toward preparing and delivering the design review and starting our Milestone 2 presentation, rather than resolving Mac access. We are comfortable with that tradeoff given those deadlines, but we are treating next week as a hard commitment point for more than just the Mac decision and project setup: we are committing to actually writing and testing real code against that setup before the week is out, not leaving coding for the week after. Our Mac-access risk in Risks and Assumptions remains open.

A.2 Week Report 8 (October 24, 2026)

1. What We Did This Past Week

This week, we tried to get Xcode set up on the department Mac without tying it to anyone's personal Apple ID, and downloaded what turned out to be Xcode 13.4.1 rather than the current release. After looking into it, we realized that version cannot actually be used for this project: Apple has required Xcode 26 or later for any App Store Connect or TestFlight upload since April 2026, and 13.4.1 is also too old to support the `@Observable` view-model pattern our own Developer Requirements chapter commits to. That sent us back to weighing our options for reliable, up-to-date Mac access: a shared or dedicated cloud-hosted Mac (paired with Xcode Cloud for automated builds), versus simply buying a personal Mac, such as Apple's new lower-cost MacBook Neo. We have not yet picked one.

2. What We Will Do Next Week

Next week, we plan to finish weighing the cloud-Mac and buy-a-Mac options, commit to one as a team, and actually begin development against it — setting up the real Xcode project and starting on our first screens and view models under the conventions in the Developer Requirements chapter.

3. Action Items

- Compare the cost and practicality of a shared cloud Mac plus Xcode Cloud, a dedicated cloud Mac, and buying a personal Mac (e.g. the MacBook Neo), for the rest of the project through May.
- Decide as a team which option to commit to.
- Once decided, set up the real Xcode project and begin implementing our first screens and view models.

4. Issues and Risks

Real implementation is now a second week behind the plan from our Week 7 report, since getting a usable, current version of Xcode turned out to be less straightforward than expected. We are treating this as a decision worth taking the time to get right rather than deferring again, and are committing to settle on one option next week rather than continuing to weigh alternatives indefinitely. Our Mac-access risk in Risks and Assumptions remains open until a dependable setup is actually in place.

A.3 Week Report 7 (October 16, 2026)

1. What We Did This Past Week

This week, we wrote our developer requirements before starting real implementation: a new Developer Requirements and Coding Standards chapter that fixes our MVVM architecture split across ScrapCore and ScrapApp, the namespace and folder structure for both modules, class and type naming conventions, and the UML diagrams (package, domain class, MVVM feature-slice, and search sequence) that lay out the app's structure. We also got the Mac our professor arranged for us working — we now have reliable access to a department

Mac instead of depending only on shared library computers, which was the risk we flagged in Week 6.

2. What We Will Do Next Week

Next week, we plan to begin real development: setting up the actual Xcode project on the department Mac, wiring the ScrapCore package into it, and building our first end-to-end slice against the conventions we just wrote down in the Developer Requirements chapter, instead of continuing to treat our code as a scaffold.

3. Action Items

- Set up the real Xcode project on the department Mac and confirm ScrapCore builds inside it.
- Begin implementing our first screens and view models following the Developer Requirements chapter's MVVM, naming, and folder conventions.
- Keep the UML diagrams in the Developer Requirements chapter current as we add the first real modules and types.

4. Issues and Risks

Now that we have the Developer Requirements chapter in place, our main risk shifts from planning to actually following it once three people are writing code at once — we are mitigating this by requiring every pull request to be reviewed against that chapter before it merges. Reliable access to the department Mac reduces, but does not eliminate, our earlier Mac-access risk; we are still tracking it in Risks and Assumptions since the arrangement is new and we have not yet relied on it for a full week of development.

A.4 Week Report 6 (October 7, 2026)

1. What We Did This Past Week

This week, we looked into whether there was a solid way to build and run our SwiftUI app without needing an actual Mac, since going in we had assumed some kind of workaround or cross-platform tool would let us use Xcode without one. After researching this, we found that there isn't a reliable way to do that — Xcode and SwiftUI genuinely only run on macOS, and our only access to a Mac is through shared Stevens library computers. Since we can't treat those library computers as something we can use constantly, we decided to split our codebase into two pieces: ScrapCore, a platform-independent Swift package holding our data models, recipe-matching logic, and networking code, which we can build and test on our own Windows machines; and ScrapApp, the SwiftUI screens themselves, which is the only part that actually needs a Mac. We used AI to scaffold both pieces following our existing requirements and the UI designs we have been commenting on, including a first pass at our core screens and a set of unit tests for the matching, grocery list, and serving-scaling logic. We also added this constraint, and our plan to minimize it, as a new risk in our report's Risks and Assumptions section.

2. What We Will Do Next Week

Next week, we plan to run and fix up the scaffolded ScrapCore package on our own machines, since none of us has compiled it yet, and schedule our first library Mac session to set up the real Xcode project and wire in the SwiftUI screens. We also still plan to look into the requirements of our operations stakeholder and research our deployment approach and Swift-specific edge cases and constraints, which we did not get to this week. In addition, we want to plan out the specifics of how we want our code itself to be written — our MVVM architecture pattern, naming conventions for namespaces and classes, and UML diagrams laying out the app's structure — so that as we start building for real, all three of us are writing consistent, compatible code rather than settling these conventions ad hoc as we go.

3. Action Items

- Run and fix up the ScrapCore package on our own Windows machines.
- Schedule and complete our first library Mac session to create the Xcode project and wire in the SwiftUI screens.
- Gather the requirements of our operations stakeholder.
- Research deployment options, including whether to use containers, and Swift-specific edge cases and constraints.
- Plan the specifics of how our code should be written: our MVVM architecture pattern, namespace and class naming conventions, and UML diagrams for the app's structure.

4. Issues and Risks

Because none of us has compiled any of this code yet, we expect to run into some build errors when we first try — we are treating that as normal first-pass work rather than a sign that our approach is wrong, and are setting aside time early next week to work through whatever comes up. More broadly, our continued reliance on shared library computers for anything involving Xcode remains a risk we are tracking in our Risks and Assumptions section. On that front, our professor is also reaching out to see if the program has a Mac we would be able to borrow, which would help reduce that reliance if it comes through.

A.5 Week Report 5 (October 1, 2026)

1. What We Did This Past Week

This week, we met as a team and sketched out our initial app screens by hand, talking through our UI design choices together as we went. We then used AI to generate sample screens and visual designs based on those sketches, and are leaving thorough comments on each generated screen noting what we want changed, removed, or added before any of it is treated as final.

2. What We Will Do Next Week

Next week, we plan to look into more logistical questions, starting with the requirements of our operations stakeholder, and to research how to actually achieve some of the features we want in practice — including our deployment approach (such as whether to containerize any

part of our backend or companion website), along with edge cases and constraints specific to building in Swift.

3. Action Items

- Finish reviewing the AI-generated sample screens and leave specific comments on each one.
- Gather the requirements of our operations stakeholder.
- Research deployment options, including whether to use containers, for our backend and companion website.
- Identify edge cases and Swift-specific constraints relevant to the features we have planned.

4. Issues and Risks

Because we are starting from AI-generated screens rather than designing every screen from scratch, we need to make sure our comments are specific and thorough enough that the final design still matches our own vision instead of drifting toward a generic look. We will address this by reviewing everyone's comments together as a team before treating any screen as finished.

A.6 Week Report 4 (September 22, 2026)

1. What We Did This Past Week

This week, we tightened up our requirements and use cases based on our Week 3 interview findings, marking the recipe video, leftover tracking, and expiration alert features as bonus rather than required, and rewriting our user stories to be more concise. We also finalized our technical plan: a native iOS app built in Swift and SwiftUI, backed by Supabase for our database, authentication, and storage, and Spoonacular for recipe and ingredient data, along with our testing approach and a plan for a companion website. We documented that plan in the report as a Development Plan chapter, along with our team roles and responsibilities, our GitHub workspace and communication practices, a risks and assumptions section citing outside research on the cost of food waste, and a timeline tied to our course milestones.

2. What We Will Do Next Week

Next week, we plan to begin implementation using our chosen stack, starting with a first end-to-end slice of the app (pantry entry connected to Spoonacular search), and continue preparing our demo presentation.

3. Action Items

- Set up the Xcode project and connect it to our Supabase backend.
- Build a first end-to-end slice: pantry entry through to a ranked recipe result.
- Continue preparing our demo presentation.

4. Issues and Risks

Because all three of us are learning Swift, SwiftUI, and Supabase at the same time, our first week of implementation may move more slowly than later weeks. We will address this by starting with a small, well-scoped first feature and checking in with each other regularly so that any one person's blockers do not stall the whole team.

A.7 Week Report 3 (September 15, 2026)

1. What We Did This Past Week

This week, we completed our first round of interviews as a group and discussed the results together to evaluate how they translate into which features we should include in our product. We also fed our interview findings into AI to generate a starting list of use cases for our project.

2. What We Will Do Next Week

Next week, we plan to tighten up our requirements and use cases based on our interview findings and the AI-generated starting list, and begin working on our demo presentation.

3. Action Items

- Refine our requirements based on the feedback gathered from our first round of interviews.
- Review and tighten up the AI-generated starting list of use cases.
- Begin preparing our demo presentation.

4. Issues and Risks

Because our interview feedback was somewhat varied, we may need additional team discussion to reach agreement on which features to prioritize. We will address this by reviewing the feedback together and aligning on priorities before finalizing our requirements and use cases next week.

A.8 Week Report 2 (September 8, 2026)

1. What We Did This Past Week

This week, we used AI to help compile and format our project report. We finished our Team Declaration and Mission Statement chapters and started drafting our Key Drivers and Key Constraints chapters, continued setting up our project documentation, and set up our GitHub repository. This work is not yet finished, and we are still working toward completing our Milestone 0 report.

2. What We Will Do Next Week

Next week, we plan to conduct more specific market research, including writing out the features we like and the features we find missing from competing products. We were not able to complete this market research last week because we spent more of our time on setting up

our project and documentation. Next week, we also each plan to conduct an interview with someone outside our team to gauge whether there is demand for our product.

3. Action Items

- Conduct specific market research on competing recipe and meal-planning applications.
- Document the features we like and the features that are missing from competitor products.
- Each team member conducts one interview to assess demand for our product.

4. Issues and Risks

Because we spent more time on project setup this past week, our planned market research was delayed by one week. To stay on schedule, we will need to complete both the delayed market research and the new interviews next week, which increases our workload for that week.

A.9 Week Report 1 (September 1, 2026)

1. What We Did This Past Week

This week, our group was formed and we selected an initial project idea. We discussed our interests and goals as a team and worked together to determine a direction for our project. We also began organizing our team and establishing a foundation for the work we will complete throughout the semester.

2. What We Will Do Next Week

Next week, we will conduct market research to gain a better understanding of our target market and the products or services related to our project idea. We will use this research to better define our product's goals, identify potential users, and understand the needs our product should address.

3. Action Items

- Conduct initial market research.
- Research similar products and competitors.
- Identify the target users for our product.
- Further define the goals and purpose of our product.
- Continue setting up our project documentation in Overleaf.

4. Issues and Risks

One issue our team is currently facing is that using LaTeX and Overleaf is new to one of our team members. To address this, the team will work together to help everyone become comfortable with the tools and ensure that all members can contribute effectively to the project documentation. We will also communicate regularly and provide assistance when needed to prevent this from becoming a larger issue as the project progresses.

Appendix B

AI Prompt Log

B.1 How to Use This Appendix

Use this log when AI assistance materially affects requirements, architecture, code, tests, or release evidence. Keep each entry to two sentences: what the AI did and why, and any correction the team still needs to make.

B.2 Log Entries

- **September 8, 2026** (Claude, Cowork). Added a Team Declaration chapter and populated Mission Statement, Key Drivers, and Key Constraints from the team's own proposal, bios, and drafted text. The team supplied all content and reviewed the compiled PDF for page limits and formatting.
- **September 10, 2026** (Claude, Cowork). Produced a trimmed submission copy containing only the chapters completed so far, as source files. The AI compiled it and reported the page count and table of contents for the team to confirm.
- **September 10, 2026** (Claude, Cowork). Added Appendix A, Weekly Reports, formatting the team's verbatim Week 1 report text into the template's four-part structure. No content was changed beyond formatting and chapter placement.
- **September 17, 2026** (Claude, Cowork). Corrected the submission label from Milestone 1 to Milestone 0 throughout, and rewrote Mission Statement and Key Drivers around the team's four core values and the social aspect of Scrap. The team supplied the correction and the core values directly.
- **September 22, 2026** (Claude, Cowork). Added User Stories, Requirements, Use Cases, and Traceability Matrix chapters, transcribed from a team-supplied preview built on the Week 3 interviews. The team supplied all substantive content; the AI handled the LaTeX structure and cross-referencing.

- **September 22, 2026** (Claude, Cowork). Reversed the Milestone 0 label back to Milestone 1, and changed REQ-F-07, REQ-F-13, and REQ-F-14 from "shall" to "could" as bonus features. Both corrections were supplied directly by the team.
- **September 22, 2026** (Claude, Cowork). Rewrote the end-user stories in "As a / I want / so that" form, removed interviewee names, added six stakeholder stories (developer, business, community moderation), and added a Technical Requirements section (REQ-T-01–07) and a content-moderation requirement (REQ-F-17). The team specified the format and stakeholder categories; the AI chose the specific stories and technical requirements, which the team should review.
- **September 22, 2026** (Claude, Cowork). Trimmed all fourteen user stories to a single sentence each, per the team's feedback that the previous rewrite was still too wordy. No requirement mappings changed.
- **September 23, 2026** (Claude, Cowork). Aligned the report with the team's development plan (Swift/SwiftUI, Supabase, Spoonacular, Open Food Facts, a companion website, and the team's testing/release approach), revising REQ-C-01/REQ-C-04 and REQ-T-01/03/04/06 and adding REQ-F-18, REQ-C-06, and REQ-T-08–13. The team supplied the complete technical plan; the AI determined how to phrase and cross-reference each requirement.
- **September 23, 2026** (Claude, Cowork). Added a Week 4 report, drafted from the session's own work tightening requirements and finalizing the technical plan since the team asked for one without dictating the text, and condensed every entry in this log to two sentences or fewer. The team should review the Week 4 report's wording for accuracy before treating it as final.
- **September 23, 2026** (Claude, Cowork). Added a Development Plan chapter after Key Constraints, summarizing the mobile application, backend, recipe data source, companion website, development method, and testing plan already detailed in Requirements. The team requested the chapter; its content is drawn entirely from the team's existing technical plan and requirements, with no new decisions made.
- **September 24, 2026** (Claude, Cowork). Added a Roles and Responsibilities section to Team Declaration, proposing a lead role per member and revising the assignments per the team's direction. The team chose the final role split; the AI drafted the supporting rationale and wording.
- **September 24, 2026** (Claude, Cowork). Replaced the Development Plan chapter's Development Method section with Workspace and Communication Plan sections, transcribing the team's own description of their GitHub workflow, shared documents, tooling, and check-in practices. The team supplied the content directly; the AI formatted it and added requirement cross-references.

- **September 24, 2026** (Claude, Cowork). Added three team-supplied sources on food waste as citations in Key Drivers and the bibliography, and added Risks and Assumptions and Timeline and Milestones sections to the Development Plan chapter, the latter modeled on the full template's official course milestone schedule. The AI drafted the risk analysis and milestone targets from the report's existing content; the team should review both for accuracy.
- **September 24, 2026** (Claude, Cowork). Revised the Timeline and Milestones section with the team's own targets: a working prototype by early December, and a spring semester centered on beta user testing, iteration, and growing an initial user base. The team supplied these targets directly; the AI wove them into the existing milestone descriptions.
- **September 24, 2026** (Claude, Cowork). Rebuilt Timeline and Milestones as two dated tables, per the team's request for two-week fall checkpoints and broader monthly spring goals with specific dates. The team set the cadence and end points (prototype by December, beta users starting in January); the AI filled in the intermediate checkpoint content and dates.
- **September 24, 2026** (Claude, Cowork). Added citations for Apple's SwiftUI documentation, the Spoonacular API, and Supabase's iOS/SwiftUI quickstart guide, verified against their live pages and cited alongside the matching technology in the Development Plan chapter. The team supplied the three links directly.
- **September 29, 2026** (Claude, Cowork). Added a Verification, Validation, and Testing chapter (a Verification Register, a Validation Register, and example test cases) covering Scrap's existing requirements, and added two bonus requirements and two use cases for photo-based pantry entry and automatic ingredient shelf-life reminders, per usability feedback the team received after presenting Milestone 1. The team supplied the feedback and the two feature ideas directly; the AI chose the specific verification methods, register rows, test cases, and use-case wording, which the team should review.
- **September 29, 2026** (Claude, Cowork). Widened the Source column in the User Stories to Requirements table so each entry fits on one line instead of wrapping. The team requested the formatting fix directly.
- **October 1, 2026** (Claude, Cowork). Added the Week 5 report to Appendix A, formatting the team's account of this week's screen sketches and AI-generated design comments, and next week's plan, into the report's four-part structure. The team supplied all of the content directly.
- **October 7, 2026** (Claude, Cowork). Added a fourth risk and mitigation to Risks and Assumptions, after the team identified that no member owns a Mac and asked how to minimize reliance on shared library computers while still building the planned Swift/SwiftUI app. The team's situation was supplied directly; the AI proposed and wrote the specific mitigation (splitting the codebase into a cross-platform ScrapCore package and a Mac-only SwiftUI shell) and the accompanying code scaffold, which the team should review.

- **October 7, 2026** (Claude, Cowork). Added the Week 6 report to Appendix A, narrating this week's finding that there is no solid way to use Xcode without an actual Mac, the ScrapCore/ScrapApp split decision, and the resulting code scaffold, into the report's four-part structure. The team supplied the events directly; the AI drafted the wording, which the team should review for accuracy.
- **October 8, 2026** (Claude, Cowork). Updated the Week 6 report's next-week plan and action items to include planning the app's MVVM architecture, namespace/class naming conventions, and UML diagrams, and added to Issues and Risks that the professor is reaching out about borrowing a Mac through the program. The team supplied both additions directly.
- **October 16, 2026** (Claude, Cowork). Added a Developer Requirements and Coding Standards chapter, drafted by the team in Overleaf, fixing the MVVM architecture split, namespace and folder structure, naming conventions, and UML diagrams for ScrapCore and ScrapApp, and added the Week 7 report covering that work and the team gaining working access to a department Mac. The team supplied the complete chapter text, diagrams, and week's events directly; the AI wired the chapter into the report and wrote the weekly report entry.
- **October 24, 2026** (Claude, Cowork). Added the Week 8 report, narrating the team's discovery that the downloaded Xcode 13.4.1 cannot be used (it predates Apple's Xcode 26+ App Store Connect requirement and does not support the `@Observable` pattern from the Developer Requirements chapter) and the team's ongoing comparison of cloud-hosted Mac options against buying a personal Mac. The team supplied the week's events directly; the AI drafted the wording, which the team should review for accuracy.

Bibliography

[1] "Estimating the Cost of Food Waste to American Consumers," U.S. Environmental Protection Agency, 2026, accessed September 24, 2026. [Online]. Available: https://www.epa.gov/land-research/estimating-cost-food-waste-american-consumers

[2] "Food Waste Data — Causes & Impacts," ReFED, 2026, accessed September 24, 2026. [Online]. Available: https://refed.org/food-waste/the-problem/

[3] C. Sideck, "A Look at Leftovers to Reduce Food Waste," U.S. Department of Agriculture, Agricultural Research Service, 2026, accessed September 24, 2026. [Online]. Available: https://scientificdiscoveries.ars.usda.gov/tellus/stories/articles/look-leftovers-reduce-food-waste

[4] "SwiftUI — Apple Developer Documentation," Apple Inc., 2026, accessed September 24, 2026. [Online]. Available: https://developer.apple.com/documentation/swiftui

[5] "Use Supabase with iOS and SwiftUI," Supabase, 2026, accessed September 24, 2026. [Online]. Available: https://supabase.com/docs/guides/getting-started/quickstarts/ios-swiftui

[6] "spoonacular Recipe and Food API," spoonacular, 2026, accessed September 24, 2026. [Online]. Available: https://spoonacular.com/food-api

[7] *Web Content Accessibility Guidelines (WCAG) 2.1*, World Wide Web Consortium Std. W3C Recommendation, 2018, accessed October 27, 2026. [Online]. Available: https://www.w3.org/TR/WCAG21/

[8] "Human Interface Guidelines," Apple Inc., 2026, accessed October 27, 2026. [Online]. Available: https://developer.apple.com/design/human-interface-guidelines

[9] "App Review Guidelines," Apple Inc., 2026, accessed October 27, 2026. [Online]. Available: https://developer.apple.com/app-store/review/guidelines/

[10] *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, Object Management Group, 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[11] *IEEE Standard for System, Software, and Hardware Verification and Validation*, IEEE Std. 1012-2024, 2025. [Online]. Available: https://standards.ieee.org/ieee/1012/7324/

[12] *ISO/IEC/IEEE 29119-2:2021, Software and Systems Engineering—Software Testing—Part 2: Test Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 29 119-2:2021, 2021. [Online]. Available: https://www.iso.org/standard/79428.html

[13] *ISO/IEC/IEEE 15288:2023, Systems and Software Engineering—System Life Cycle Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 15 288:2023, 2023. [Online]. Available: https://www.iso.org/standard/81702.html

[14] *ISO/IEC/IEEE 12207:2026, Systems and Software Engineering—Software Life Cycle Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 12 207:2026, 2026. [Online]. Available: https://www.iso.org/standard/90219.html

[15] *ISO/IEC/IEEE 29148:2018, Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, International Organization for Standardization Std. ISO/IEC/IEEE 29 148:2018, 2018. [Online]. Available: https://www.iso.org/standard/72089.html

Glossary

acceptance criterion An observable condition used to decide whether a need, story, or requirement has been satisfied. 69

Could A desirable capability included only when time and resources permit. 69

Must A capability required in the current release; the release is unacceptable without it. 69

requirement A necessary, singular, unambiguous, feasible, and verifiable statement of capability or constraint. 69

Should An important capability expected in the current release unless a documented constraint prevents it. 69

stakeholder A person, group, or organization that affects the system or is affected by it. 69

traceability The ability to follow an artifact backward to its source and forward to its implementation and evidence. 69

use case A goal-oriented interaction between an actor and the system, including normal and alternate flows. 69

V-model A lifecycle model that pairs each decomposition and specification activity with a corresponding integration and evidence activity. 69

validation Confirmation that the completed system satisfies stakeholder needs in its intended context; building the right system. 69

verification Confirmation that specified requirements have been fulfilled; building the system right. 69

Would A capability assigned to a later implementation version after higher-priority work. 69

Acronyms

AI artificial intelligence. 69

UI user interface. 69

UML Unified Modeling Language. 69

V&V verification and validation. 69

Index

activity diagrams, 28
AI prompt log, 64
architecture
    MVVM, 42

Chapter
    AI Prompt Log, 64
    Codes and Standards, 53
    Developer Requirements, 42
    Development Plan, 9
    Key Constraints, 7
    Key Drivers, 5
    Mission Statement, 4
    Requirements, 17
    Team Declaration, 2
    Traceability Matrix, 31
    Use Cases, 23
    User Stories, 14
    Verification, Validation, and Testing, 36
    Weekly Reports, 57
codes and standards, 53

design iteration, 8
development plan, 9
document history, iv

key constraints, 7
    tradeoffs, 8
key drivers, 5

mission statement, 4

namespaces, 44
naming conventions, 46

Requirement
    REQ-C-01, 20
    REQ-C-02, 20
    REQ-C-03, 20
    REQ-C-04, 20
    REQ-C-05, 21
    REQ-C-06, 21
    REQ-D-01, 43
    REQ-D-02, 43
    REQ-D-03, 43
    REQ-D-04, 44
    REQ-D-05, 44
    REQ-D-06, 44
    REQ-D-07, 44
    REQ-D-08, 44
    REQ-D-09, 44
    REQ-D-10, 45
    REQ-D-11, 45
    REQ-D-12, 46
    REQ-D-13, 46
    REQ-D-14, 46
    REQ-D-15, 46
    REQ-D-16, 47
    REQ-D-17, 47
    REQ-D-18, 48
    REQ-D-19, 48
    REQ-D-20, 48
    REQ-D-21, 48
    REQ-D-22, 49
    REQ-D-23, 49
    REQ-D-24, 49
    REQ-D-25, 49
    REQ-D-26, 50
    REQ-F-01, 17
    REQ-F-02, 17
    REQ-F-03, 17
    REQ-F-04, 17
    REQ-F-05, 17
    REQ-F-06, 18
    REQ-F-07, 18

REQ-F-08, 18
REQ-F-09, 18
REQ-F-10, 18
REQ-F-11, 18
REQ-F-12, 18
REQ-F-13, 18
REQ-F-14, 18
REQ-F-15, 19
REQ-F-16, 19
REQ-F-17, 19
REQ-F-18, 19
REQ-F-19, 19
REQ-F-20, 19
REQ-NF-01, 19
REQ-NF-02, 20
REQ-NF-03, 20
REQ-NF-04, 20
REQ-NF-05, 20
REQ-T-01, 21
REQ-T-02, 21
REQ-T-03, 21
REQ-T-04, 21
REQ-T-05, 21
REQ-T-06, 22
REQ-T-07, 22
REQ-T-08, 22
REQ-T-09, 22
REQ-T-10, 22
REQ-T-11, 22
REQ-T-12, 22
REQ-T-13, 22
requirements, 17
constraints, 20
developer, 42
functional, 17
non-functional, 19
technical, 21

standards
process, 53
product-facing, 54

team declaration, 2
TestCase
TC-001, 40
TC-002, 40
TC-003, 40
TC-004, 41
TC-005, 41
TC-006, 41
testing, 36
test cases, 40
traceability matrix, 31

UML, 49
activity diagrams, 28
use cases, 23
UseCase
UC-01, 23
UC-02, 23
UC-03, 24
UC-04, 24
UC-05, 24
UC-06, 25
UC-07, 25
UC-08, 26
UC-09, 26
UC-10, 26
UC-11, 27
UC-12, 27
user stories, 14
UserStory
US-01, 15
US-02, 15
US-03, 15
US-04, 15
US-05, 15
US-06, 15
US-07, 15
US-08, 15
US-09, 16
US-10, 16
US-11, 16
US-12, 16
US-13, 16
US-14, 16

Validation
VAL-001, 39
VAL-002, 39

VAL-003, 39
VAL-004, 39
VAL-005, 39
VAL-006, 39
VAL-007, 39
VAL-008, 39
VAL-009, 40
VAL-010, 40
validation, 36
register, 38
Verification
VER-001, 36
VER-002, 36
VER-003, 37
VER-004, 37
VER-005, 37
VER-006, 37
VER-007, 37
VER-008, 37
VER-009, 37
VER-010, 37
VER-011, 38
VER-012, 38
VER-013, 38
VER-014, 38
VER-015, 38, 55
verification, 36
register, 36

weekly reports, 57